

C语言与程序设计

The C Programming Language

王多强

QQ: 1097412466

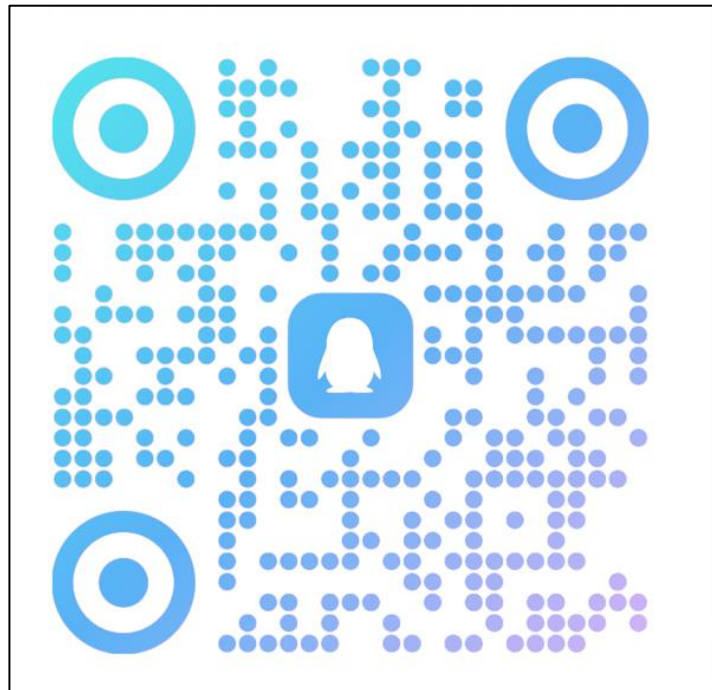
dqwang@hust.edu.cn

华中科技大学计算机学院

◆ **QQ群号**（答疑、通知）：

群名称：C语言-2025秋

群 号：729866663



◆ **微助教**（点名、课件）：

课堂名称：C语言-2025秋

课堂编号：QB936



◆ 头歌 (<https://www.educoder.net/>)

(线上实验、理论课作业)

课堂名称: C语言程序设计2025秋

邀 请 码: 2501班: Y69LMV

2502班: 9QVX3N

2503班: SZPAQV

 实训作业	18
理论课编程作业	10
实验课实验内容	8
 理论课普通作业	8

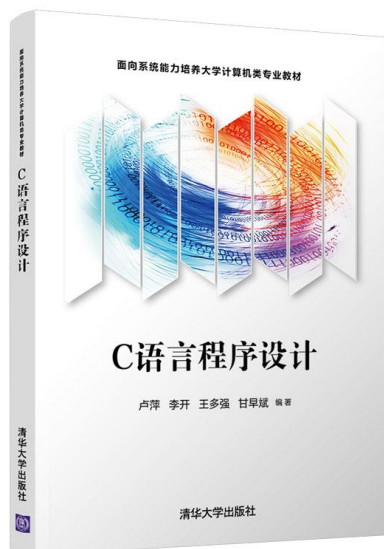
都要做

从现在起就可以做题了, 2026年1月10日16:00 结束

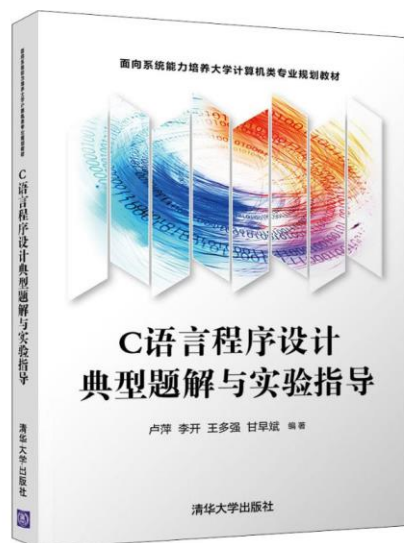
■ 课程体系：理论+实验：

- **理论课**：C语言基础，程序设计、算法设计基础
- **实验课**：编程实践，通过实践训练，熟练掌握C程序设计的方法、技术、技巧

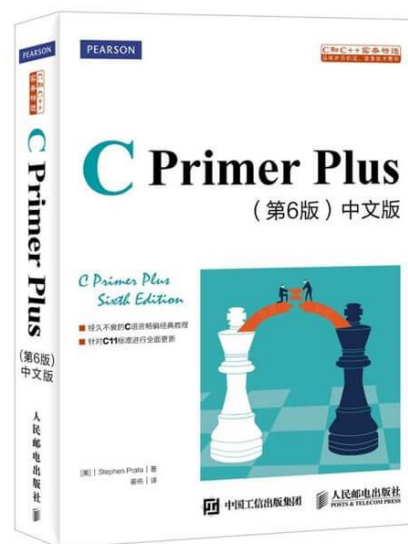
□ 理论课教材：



□ 实验课教材：



□ 参考书：



■ 理论课：48学时

- ◆ 平时成绩30%：作业、课堂提问、课堂表现等
- ◆ 考试成绩70%：闭卷考试，卷面成绩

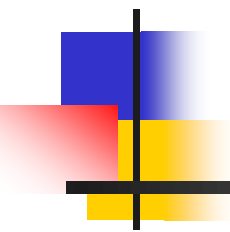
■ 实验课：32学时

- ◆ 刷题，三部分题目都要做
- ◆ 撰写实验报告

■ 考勤

- ◆ 微助教：每次上课签到，20秒；

华中科技大学公众号 -> 应用-> 微助教



C语言与程序设计

The C Programming Language

第1章 概论

本章主要内容



1.1 计算机发展简史

1.2 计算机系统概述

1.3 程序设计语言与程序设计

1.4 C语言的产生、发展与语言特征

1.5 着手编程：从第一个程序做起

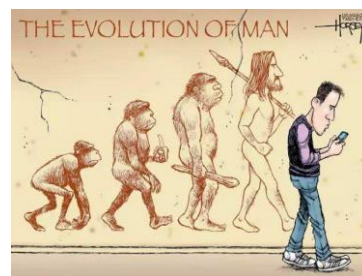
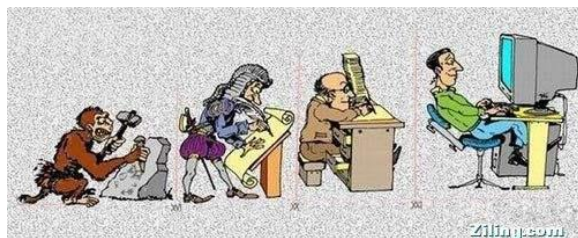
1.6 如何使用计算机进行问题求解

1.7 算法及其表示

1.1 计算机发展简史

电子数字计算机是20世纪人类最伟大的发明之一。

数字计算机从诞生到广泛应用，时至今日，已深刻改变了我们的世界。如今计算机成为我们工作、学习和生活中不可或缺的一部分。



1. 人类计算的梦想

人类对计算的追求是孜孜不倦的。计算的方法、技术和工具持续发展，是人类文明进步和科技发展的重要基础。

Digit 这个单词是什么意思？

◆ “数字”、“手指”



远古时代：掰手指，垒石块

新石器时代：“结绳”计数



近 古 代:

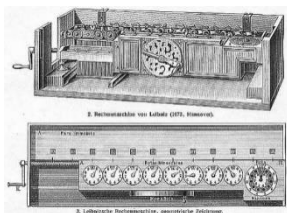
- 人类最早有实物作证的**计算工具**诞生在中国：**算筹**
 - ◆ 出现于中国商周时期
 - ◆ **祖冲之使用算筹计算圆周率**
 - ◆ “运**筹**帷幄之中，决胜千里之外”
- 中国古代的手工“计算机”：**算盘**
 - ◆ 产生于汉代，成型于南北朝
 - ◆ 距今1500年



2. 计算进入机械时代（十七世纪，欧洲科技革命）

人类对于**计算机器**的梦想一直没有间断。

- ◆ **加法器**：1642年，法国科学家**布莱斯·帕斯卡**(Blaise Pascal)发明



第一次确立了计算机器的概念

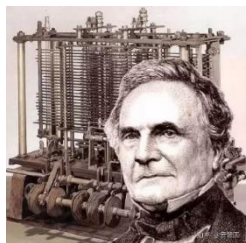
- ◆ **乘法器**：1674年，德国数学家**莱布尼茨**(Gottfried Wilhelm Leibniz)发明。



莱布尼茨, G. W.

莱布尼茨受中国“易图”（八卦）的启发，最终悟出了**二进制数**的真谛。

- ◆ **差分机**：18世纪末，英国剑桥大学教授**巴贝奇** (Charles Babbage) 发明



能够自主完成三组十万以内的加法，而不需要任何人力计算

巴贝奇被视为现在计算机的奠基人

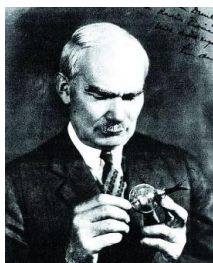
3. 电子数字计算机的诞生

1) 二十世纪初，电子文明的曙光：电子二极管、三极管

- ◆ **真空二极管**：1904年，英国电机工程师约翰·弗莱明发明



- ◆ **真空三极管**：1906年，美国人李·德·福雷斯特发明



电子管的诞生为通讯、广播、电视等技术的发展铺平了道路。也为**数字电子计算机的发明奠定了基础。**

2) 第一台电子数字计算机的诞生——ENIAC

1946年2月14日，世界第一台电子计算机在美国宾夕法尼亚大学诞生，发明人是美国人**莫希利**和^(他的学生)**埃克特**，用于**炮弹弹道轨迹计算**。

ENIAC：电子数值积分和计算机，Electronic Numerical Integrator And Computer，**埃尼阿克**，(1943-1946)。



- ◆ ENIAC总共安装了**17468只电子管**，7200个二极管，70000多电阻器，10000多只电容器和6000只继电器。**占地面积170平米**，总重量30吨，耗电174千瓦。
- ◆ ENIAC的运算速度：**每秒5000次加法**，可在0.003秒内完成两个10位数乘法。

ENIAC有多快呢？

以圆周率(π)的计算为例，

- ◆ 中国古代科学家祖冲之利用算筹，耗费15年心血，才把圆周率计算到小数点后7位数。一千多年后，英国人香克斯以毕生精力计算圆周率，计算到小数点后707位。
- ◆ 而ENIAC仅用了40秒就达到了这个记录，还发现了香克斯的数据中第528位是错误的。
- ◆ 解决复杂的弹道计算问题，由相比当时最快的计算方法，由20分钟缩短到30秒，**速度提高40倍。**

ENIAC标志着电子计算机的创世，人类社会从此迈入电子计算机新时代。—— “电脑” (英国无线电工程师协会**蒙巴顿**将军)

现在历史公认：真正的第一台电子计算机是**阿塔纳索夫-贝瑞**计算机，简称**ABC**计算机。诞生在1937年，爱荷华州立大学，不可编程，仅用于求解线性方程组。

1973年，美国联邦地方法院注销了ENIAC的专利，并得出结论：ENIAC的发明者（莫希利）从阿塔纳索夫那里继承了电子数字计算机的主要构件思想。因此，ABC被认定为世界上第一台计算机。

◆ 但ENIAC并不是完整意义下的现代计算机。

- 采用**十进制**而非二进制
- 程序采用**连线**方式实现
- **非冯·诺依曼体系结构**

◆ 冯·诺依曼体系结构的诞生

- 冯·诺依曼，美籍匈牙利数学家，被誉为**现代计算机之父**。

1944年夏，冯·诺依曼偶遇美国弹道实验室的军方负责人戈尔斯坦，受邀请加入ENIAC的研制组，开启了现代计算机研究的新纪元。随后提出了新机器设计方案，该方案被命名为**EDVAC**（**E**lectronic **D**iscrete **V**ariable **A**utomatic **C**omputer，**离散变量自动电子计算机**）。

◆ “101报告”

冯·诺依曼于1945年6月发表了著名的101页报告“关于EDVAC的报告草案”（en:First Draft of a Report on the EDVAC），明确奠定了新机器由五个部分组成，包括：**运算器、控制器、存储器、输入设备和输出设备**。报告提出的体系结构一直延续至今，被称为**冯·诺伊曼体系结构**。

◆ 现代计算机的工作原理：“**存储程序**”工作原理

把程序本身当作数据来对待，程序和该程序处理的数据用同样的方式首先储存在机器里，机器启动后应该自动地按照程序顺序取出和执行，不需要人工干预。

计算机的数制采用**二进制**。

◆ 冯·诺依曼体系结构的基本特征

- 计算机由**运算器**、**控制器**、**存储器**、**输入设备**和**输出设备**五大基本部件组成；
- 采用“**存储程序**”工作方式；
- 采用**二进制**形式表示指令和数据。

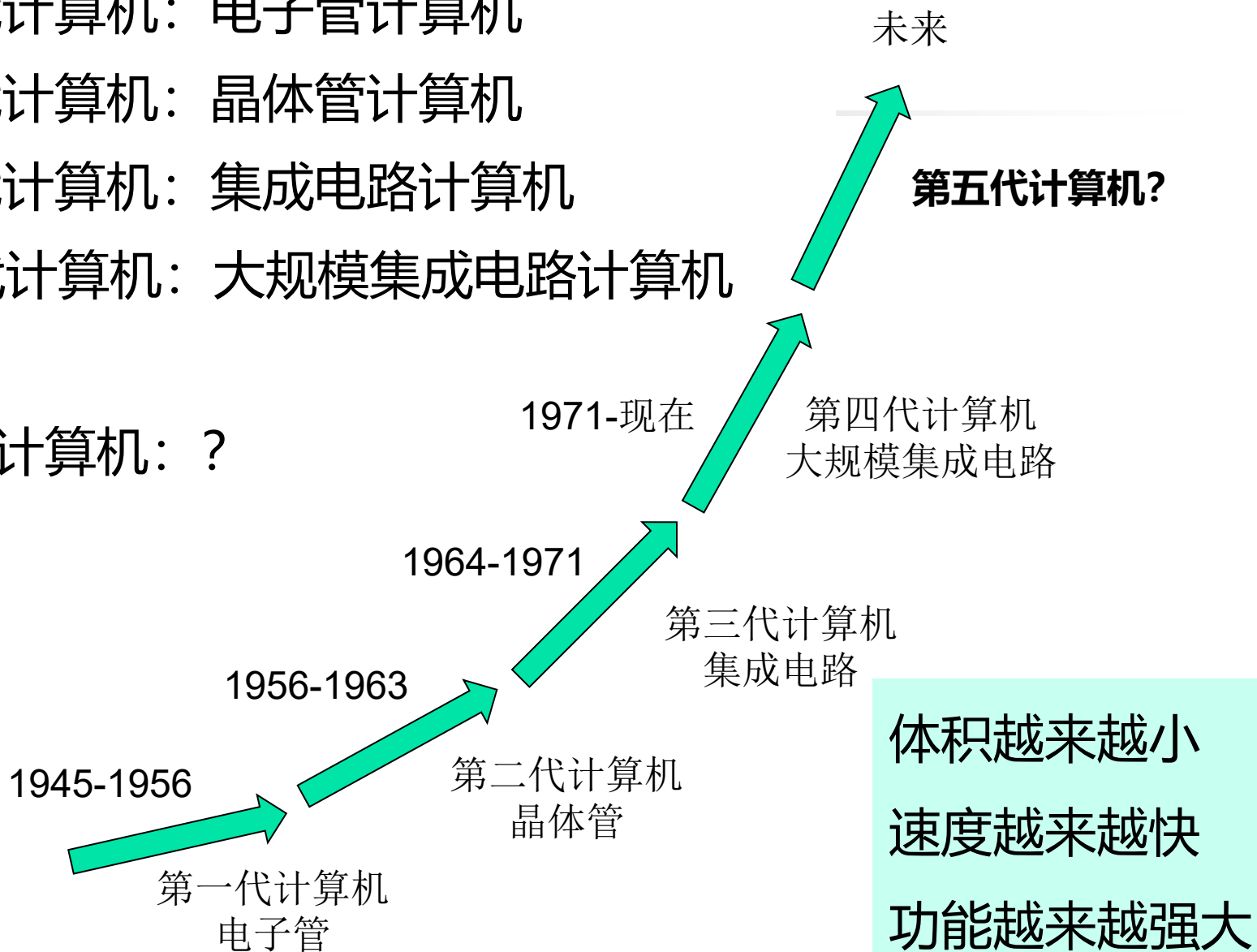
1946年，普林斯顿高等研究院（the Institute for Advance Study at Princeton, IAS）开始设计“存储程序”式计算机，被称为**IAS计算机**，但这台计算机1951年才完成，使得它并不是世界上第一台存储程序计算机。世界上第一台存储程序计算机是1949年由英国剑桥大学完成的**EDSAC**。

冯·诺伊曼结构中各基本部件的功能是：

- **存储器：**存放数据和指令，两者形式上没有区别（二进制编码），但计算机应能区分是数据还是指令；
- **控制器：**能够自动取出指令并控制指令按顺序依次执行；
- **运算器：**进行基本的算术运算、逻辑运算和其它附加运算；
- 操作人员（外界）可以通过**输入设备、输出设备**和主机进行通信。

4. 现代电子计算机的发展历程

- 第一代计算机：电子管计算机
- 第二代计算机：晶体管计算机
- 第三代计算机：集成电路计算机
- 第四代计算机：大规模集成电路计算机
- ...
- 第N代计算机：？



1) 第一代电子管计算机

- ◆ 以ENIAC、 EDSAC等为代表

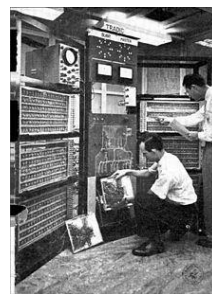
- ◆ **特点：**

- ▣ 计算机的元器件采用**电子管**
- ▣ 操作指令是为特定任务而编制，功能受限
- ▣ 软件采用**机器语言、汇编语言**编程
- ▣ 体积庞大、价格昂贵
- ▣ 仅为少数专门领域的专业人员使用

2) 第二代晶体管计算机

1947年12月23日，美国科学家巴丁、布莱顿、肖克莱发明了科技史上具有划时代意义的成果——**晶体管**，推动了全球范围内的**半导体电子工业**的发展，集成电路、大规模集成电路应运而生，电子计算机的发展也进入了快速发展的道路。

- ◆ 1954年美国电报电话公司（AT&T）贝尔实验室研制成功第一台半导体计算机**TRADIC**；
- ◆ 1960年IBM全面推出晶体管化的**7000系列电脑**，成为第二代电子计算机的典型代表

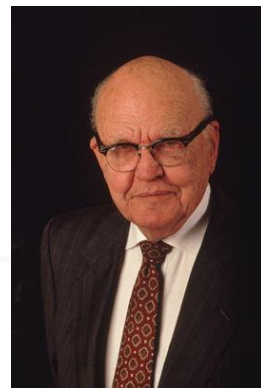


◆ 特点：

- 计算机的元器件采用**晶体管**，使用**磁芯存储器**
- 出现了**COBOL**、**FORTRAN**等高级程序设计语言
- 出现了操作系统
- 体积大为缩小，功耗低，性能、可靠性大幅提高
- 出现了程序员、分析员、计算机系统专家等新职业
- 诞生了软件产业

3) 第三代集成电路计算机

1958年9月12日，美国工程师**杰克·基尔比**成功地把20多个电子器件集成在一块面积不超过4平方毫米的半导体材料上，这一天，被视为**集成电路的誕生日**。



42年后的2000年，基尔比因发明集成电路被授予**诺贝尔物理学奖**

- 当时的**仙童半导体公司**迅速做出反应，开始生产制造计算电路。
- 1964年，仙童公司的**戈登·摩尔**（G.Moore）博士发表了一个奇特的理论：**集成电路上能被集成的晶体管数目，将会以每18个月翻一番的速度稳定增长，并在今后数十年内保持着这种势头**。摩尔的预言在后来集成电路的发展史上得以证实，被人誉为“**摩尔定律**”。

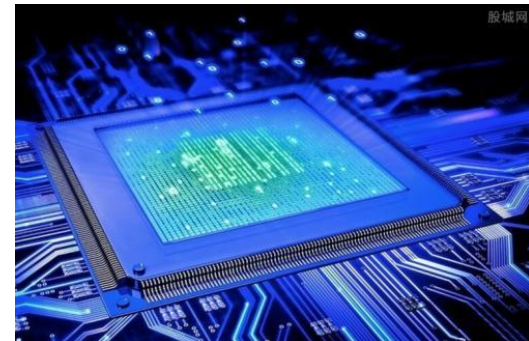


◆ 特点：

- 计算机的元器件采用**中小规模集成电路**，主存储器采用**半导体存储器**。
- 运算速度可达每秒几十万次至几百万次。
- 分时操作系统
- 高级语言有了新的发展：CPL、**BCPL**、PASCAL
- 体积更小，功耗更低，可靠性更高，应用领域更广。

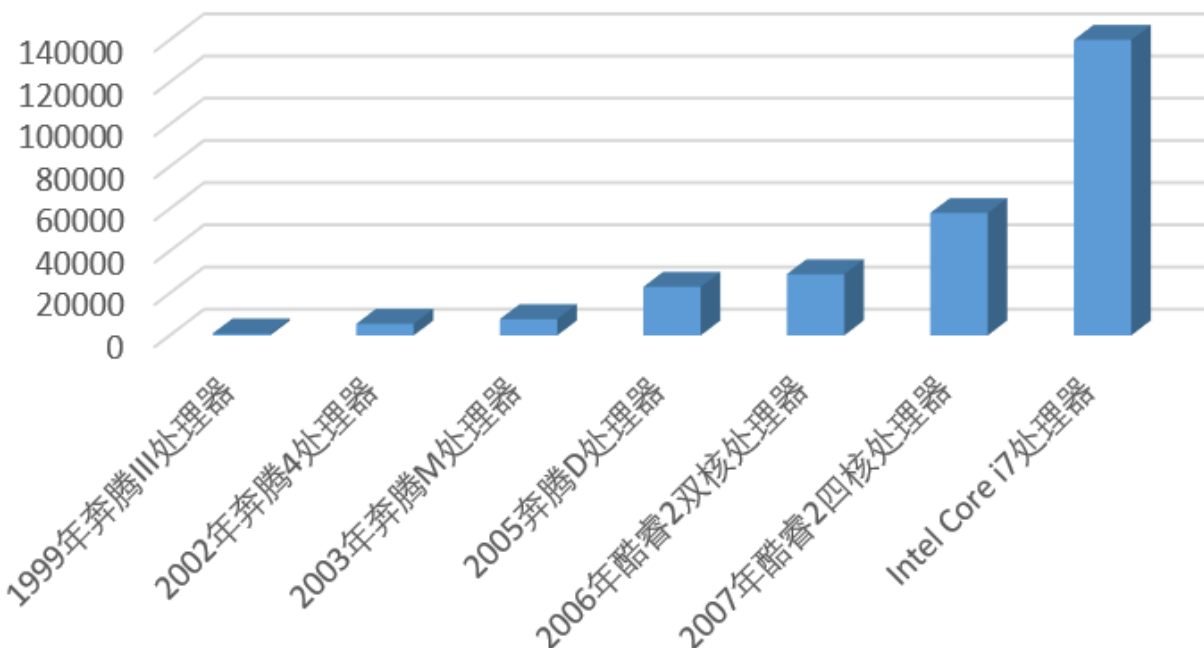
4) 第四代大规模集成电路计算机

- ◆ 小规模集成电路 (SSI) : 10-100个元件 (晶体管数量)
- ◆ 中规模集成电路 (MSI) : 100-1000个元件
- ◆ 大规模集成电路 (LSI) : 1000个以上的元件
- ◆ 超大规模集成电路 (VLSI) : 10万+个元件
- ◆ 特大规模集成电路 (ULSI) : 1000万+个元件
- ◆ 巨大规模集成电路 (GSI) : 1亿+个元件



Intel CPU芯片的集成度

Intel CPU芯片集成度(万个晶体管)



1999年奔腾III: 950万

2002年奔腾4: 5500万

2003年奔腾M: 7700万

2005年奔腾D: 2.3亿

2006年酷睿2双核: 2.9亿

2007年酷睿2四核: 5.8亿

Intel Core i7处理器: 14亿

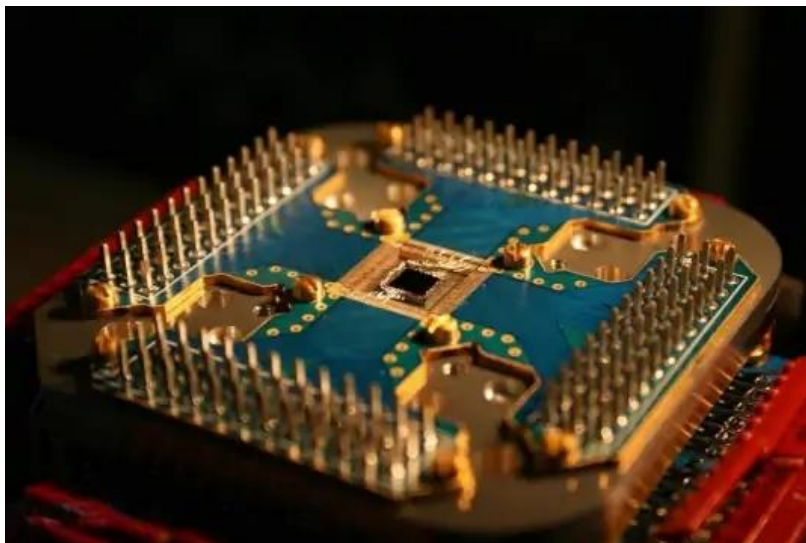
现代: 千亿级

◆ 特点：

- 计算机的元器件采用**大规模（LSI）**、**超大规模集成电路（VLSI）**，**新一代存储器**。
- 运算速度大大提高，功耗大大降低，可靠性大大提高。
- **微型化**，个人电脑走进千家万户和人们的日常生活，人类正式进入计算机时代。

未来计算机的发展方向？

- ◆ 分子计算机
- ◆ 生物计算机
- ◆ 光子计算机
- ◆ 纳米计算机
- ◆ **量子计算机**



电子数字计算机的诞生与发展是人类智慧的结晶

1.2 计算机系统概述

1.计算机系统的组成

你看到的计算机长什么样？



台式机



笔记本电脑



智能手机

■ 一般计算机系统由主机和外部设备两大部分组成

◆ 主机：

主机的关键构成部件：



台式机主机箱的内部



主板



CPU



硬盘



内存条



显卡



电源



CPU风扇/散热器

◆ 外部设备：

输入设备：采集外部世界的信息，转换成计算机可处理的数据并输入计算机



键盘



鼠标



摄像头



扫描仪

输出设备：将计算机内部的数据，以特定的形式展示出来，为外部所使用



平板显示器
(液晶、LED)



CRT显示器



激光打印机



针式打印机



绘图仪

2. 冯·诺依曼体系结构

冯·诺依曼的“关于EDVAC的报告草案”给出了**现代计算机的体系结构**，由五个部分组成：**运算器、控制器、存储器、输入设备、输出设备**。这一体系结构被称为**冯·诺伊曼结构**。

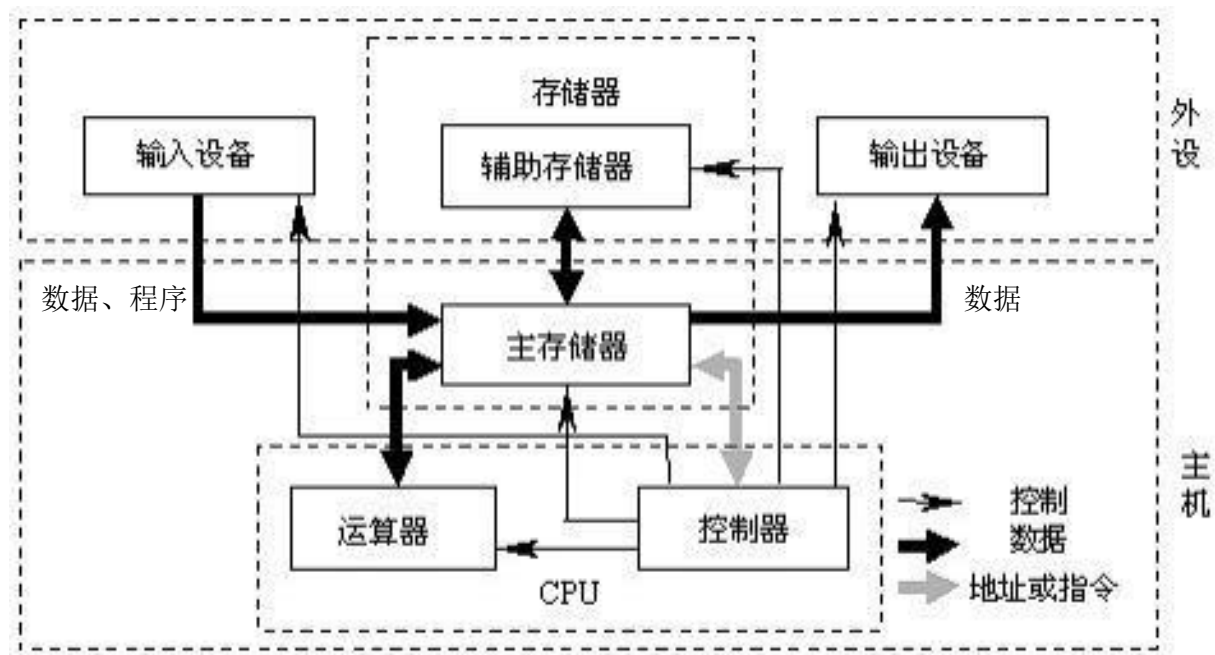


图 1-2 计算机的组成框图

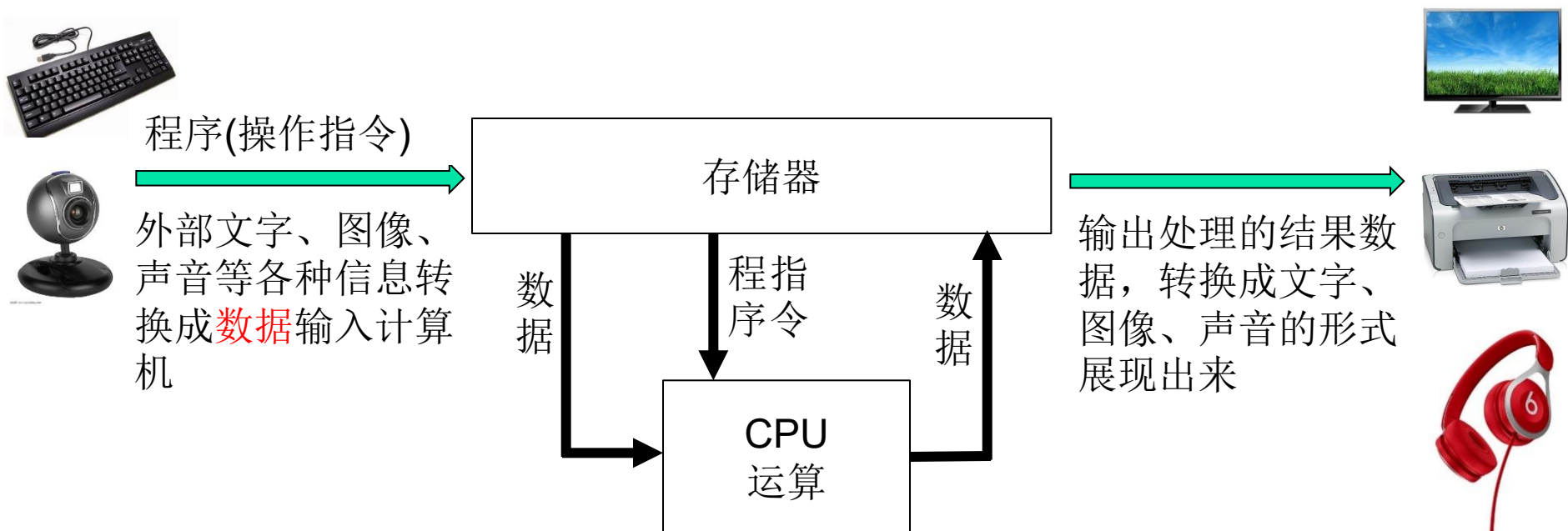
CPU：中央处理器

(Central Processing Unit)

执行程序，对数据
进行运算

Memory：存储器

存储数据和程序



运算器：执行运算

运算：算术运算（加减乘除）、逻辑运算（与或非）

算术逻辑单元(ALU, arithmetic and logic unit)

CPU：运算器 + 控制器

控制器：控制计算机的内部运行（控制指令、数据的存、取、执行）

存储器

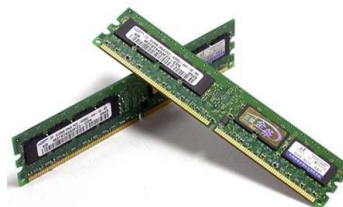
- 存储器分**内存**和**外存**

- **外存**：硬盘（包括SSD）、U盘、光盘等；

可以**长期存储数据和程序**，即使关机也不会丢失；
存放在里面的数据和程序以**文件**的形式存在。

- **内存**：插在主板上的内存条；

只用于机器运行期间**暂时存放数据和程序**，关机后丢失；
在OS的管理下，数据和程序被存放在内存空间里，等待
处理或执行。



1.3 程序设计语言与程序设计

1. 程序设计语言

自然语言（中文、英文等）是人与人之间交流的工具，是一种有规则的结构。交流的时候，能够被人理解并传递信息。



人与计算机如何打交道？

人与计算机、计算机与计算机之间传递信息，也需要语言，称为**计算机语言**。

其中，用于书写计算机程序、实现人与计算机交互的语言称为**程序设计语言**。

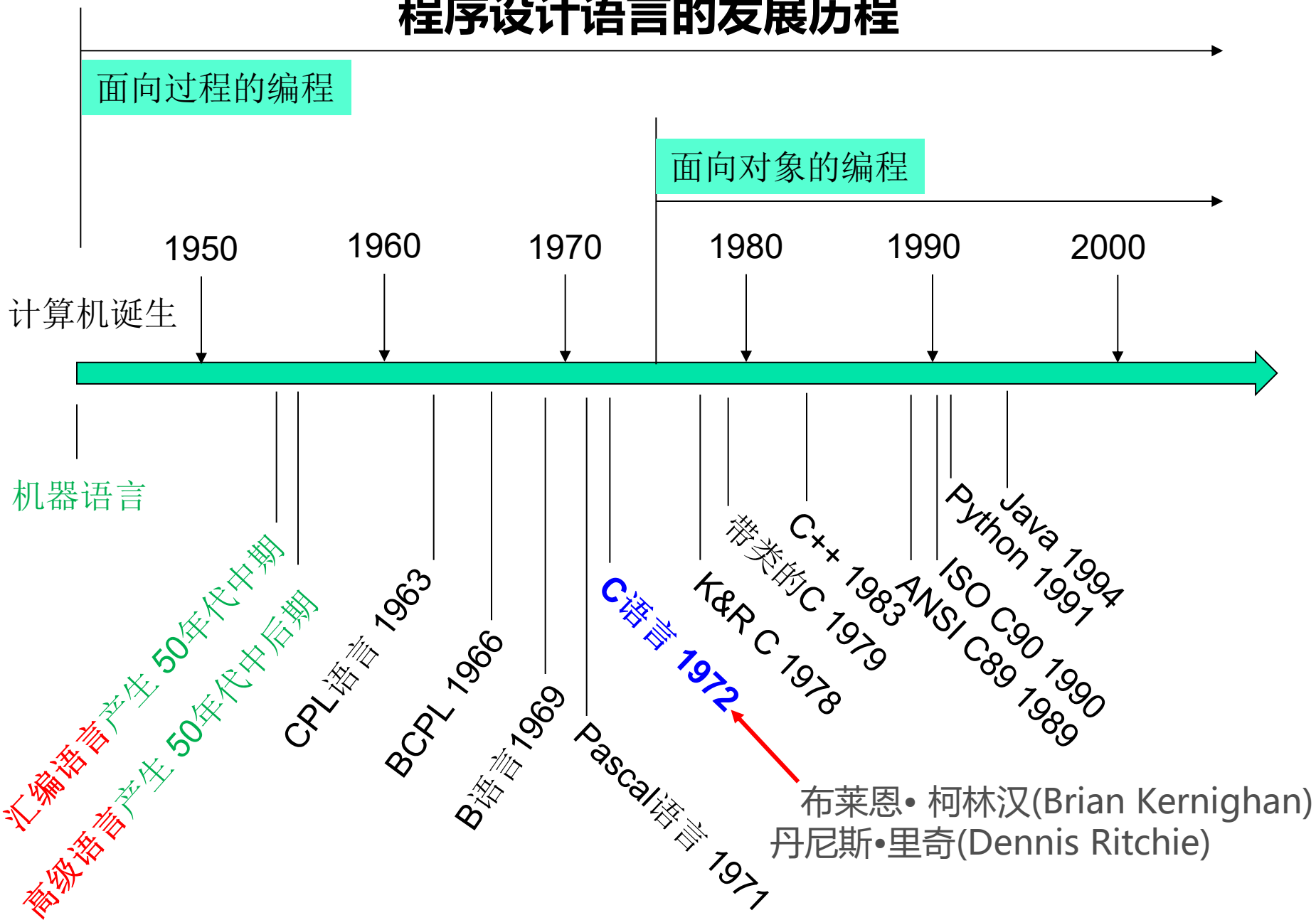


程序设计语言是以具有特定语义的符号为基本构成单位、以语法为程序构成规律，专门用于定义、组织计算机指令以完成各种各样**计算任务**的语言体系。

	自然语言	程序设计语言
特定语义的符号	字、词、各种符号	关键字、标识符、运算符
以语法为程序构成规律	组词、造句	语句
定义、组织、完成各种各样的计算任务	写文章	编程序

编程序就是用程序设计语言写一篇“计算机能读懂的文章”

程序设计语言的发展历程



FORTTRAN语言是世界上第一个被正式推广和使用的高级语言，1956年开始正式使用

计算机语言三大类型：机器语言、汇编语言、高级语言

1) **机器语言**：是用**二进制**代码表示的、计算机能直接识别并执行的**机器指令**的集合。

最早的计算机程序都采用机器语言来编写。

机器指令：是长度有限的用特定格式表示的0-1数码序列。

如： 1011 1000 0011 0100 0001 0010
操作码 w 目的 源

机器指令的基本组成：**操作码**：指出指令的功能，即做什么；

MOV AX,1234H

源寄存器/操作数：指令操作的对象；

目的寄存器/地址：操作结果放在那里；

机器语言的特点：

- 机器指令是用**0**和**1**组成的一串代码（二进制代码）；
- 机器语言程序直接用**二进制代码**指令表示；
- 不用翻译，计算机可以直接执行；
- 执行效率高。

机器语言的缺点：

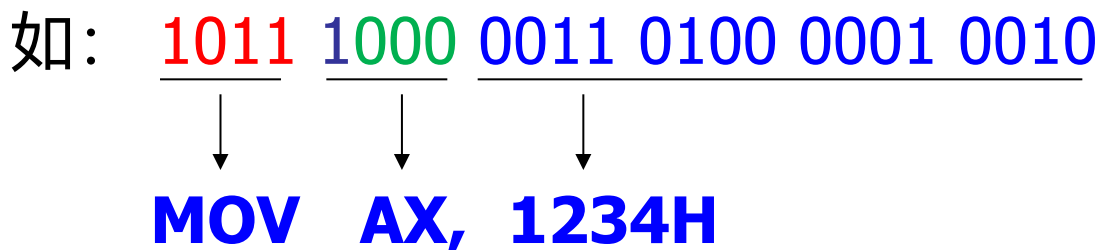
- **繁琐、难以记忆、难以编写**：只适用于非常专业的人士
- **没有数据类型的概念**：任何数据都被表示成“位串”，
不适合面向应用的编程

2) **汇编语言**：解决机器语言难以记忆和书写的问题

助记符：帮助记忆的符号，用名称表示指令、寄存器等；

这些名字是见简单的**单词缩写**或**命名字符串**。

汇编语言用**助记符**表示指令名和指令中的源或目的寄存器等。

如：

MOV AX, 1234H

优点：使用助记符，**方便记忆**，提高了编程效率

缺点：与机器语言一一对应，可视为是“**符号化了的机器语言**”，本质上还是低级语言，用汇编语言编写程序依然很繁琐。

用**汇编语言**编写的程序称为**汇编语言程序**。

汇编语言程序不能直接执行，需要经过**汇编程序**“**汇编**”后，翻译成机器语言程序才能为被计算机执行。

注：只有机器语言程序才能被计算机直接执行。

机器语言和汇编语言都是**低级语言**

- ◆ 面向机器而不是面向人的
- ◆ 没有数据类型的概念
- ◆ 对客观世界的问题描述能力差

3) **高级语言**：更方便“人”使用

高级语言是用接近于**数学和自然语言**的方式表示计算机指令的程序设计语言。

特点：

- 机器语言和汇编语言是面向机器的，而高级语言是面向用户、面向应用的；
- 方便人编写程序；
- 引入了**数据类型**的概念，对客观世界的描述能力强；
- 高级语言程序也不能被计算机直接执行，要经过翻译（**编译或解释**），转换成机器语言程序才能被计算机执行。

C、PASCAL、FORTRAN都属于高级语言。

2. 程序与程序设计

1) **程序**：就是用程序设计语言表示的计算机解题算法或解题任务，是计算机指令的集合。

2) **程序设计**：就是将解题任务转变成程序的过程。

- ◆ **程序**是计算机要执行的**指令序列**，用程序设计语言描述。
- ◆ **程序设计**是计划、安排计算机必须遵循的操作步骤及顺序的过程。

本课程将学习使用一种高级程序设计语言——C语言进行程序设计的技术。

1.4 C语言的产生、发展与语言特征

1.4.1 C语言的产生与发展

1946年，世界上第一台通用电子计算机 **ENIAC** 诞生，用机器语言进行编程。

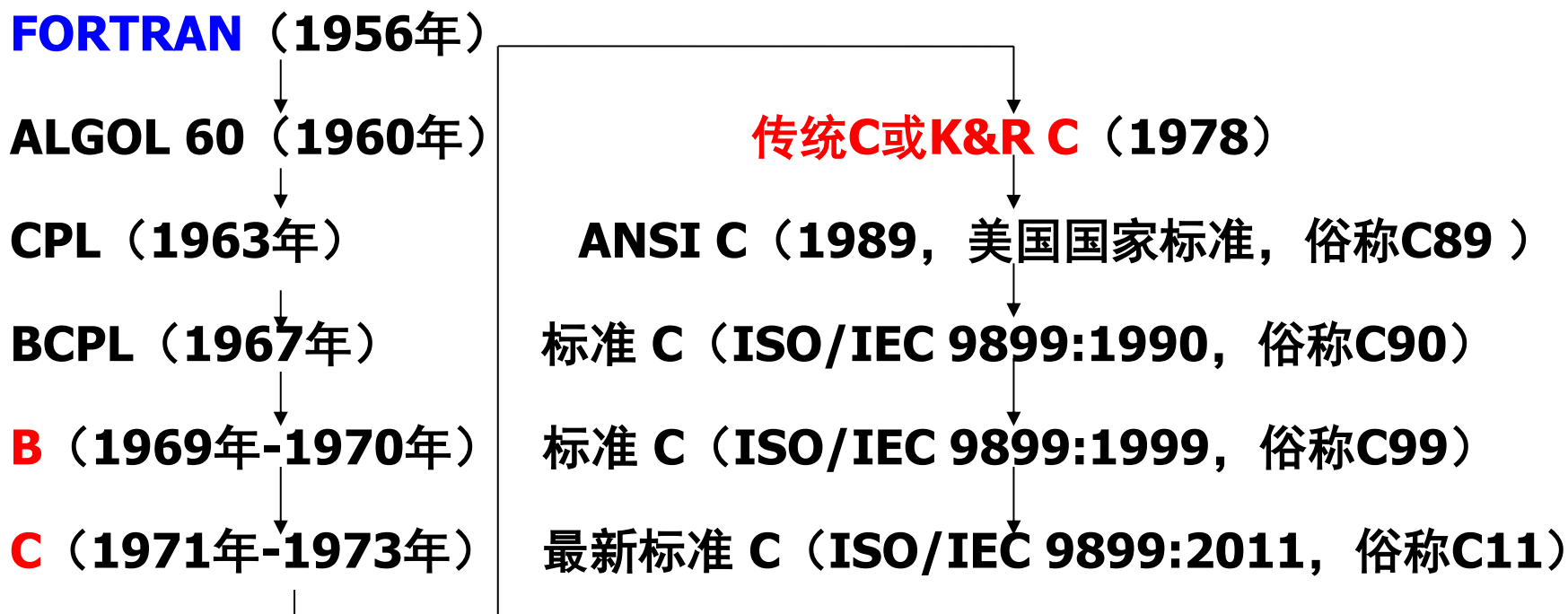


图1.1 C语言的继承、产生与发展历程



Martin Richards
1967年，剑桥大学，
BCPL编程语言的发明者



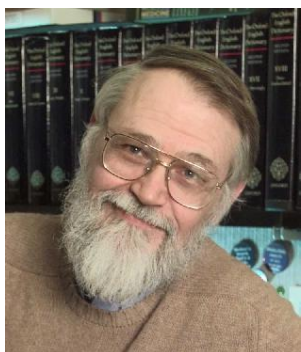
Kenneth L. Thompson
1969年，美国贝尔实验室
B语言的发明者



— Dennis M. Ritchie (丹尼斯·里奇)
在B语言的基础上设计了C语言，并使用C语言设计实现了UNIX操作系统，而被称为**C语言之父**、**UNIX之父**。1983年获得了图灵奖。



Kenneth L. Thompson (肯尼斯·汤普森)
1973年与Dennis M. Ritchie合作用C语言重写了UNIX操作系统内核。



Brian W. Kernighan (布莱恩·柯林汉)
1978年与Dennis M. Ritchie合著了巨著《**C Programming Language**》（C语言圣经），是世界上第一本被广泛认可的C语言教程，该书介绍的C语言被称为**K&R C**（或**传统C**）。

1.4.2 C语言的标准化

- 以1978年**K&R C**为代表的C语言被称为**传统的C语言**。
- 1989年底ANSI公布美国第一个C语言的国家标准 ANSI 89，简称**C89**。
- 1990年，国际标准化组织ISO接受ANSI 89为C语言的国际标准，称为ISO/IEC 9899-1990。它是C语言的第一个国际标准，也称为标准C，简称**C90**。
- ISO/IEC在1995年公布了一个新的C语言标准草案，称为**C95**，供讨论和征求意见。接着ISO/IEC在1998年又公布新标准的草案WG14/N843和WG14/N897，进一步就C语言标准的完善征求意见。
- ISO/IEC于1999年12月公布了C语言国际标准ISO/IEC 9899:1999 (E)。它是C语言国际标准9899的第二版，俗称**C99**。
- ISO/IEC于2011年12月公布了C语言国际标准ISO/IEC 9899:2011。它是C语言国际标准9899的第三版，俗称**C11**。

1.4.3 C语言的特征

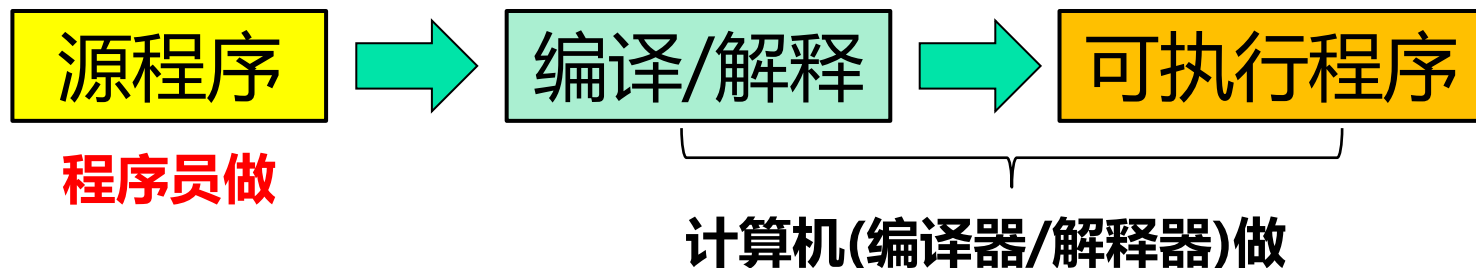
- 1) **语言简洁紧凑**：短小精悍，`while(*s++=*t++)`;
- 2) **目标代码质量高**：是目前公认的执行效率最高的高级语言；
- 3) **语言表达能力强**：数据类型丰富，操作符多、语句集简单且功能强大；
- 4) **流程控制结构化**：结构化程序设计；面向过程的编程；
- 5) **弱类型**：不同类型的数据可以有限相互转化，使用方便；
- 6) **中级语言特性**：具备对计算机硬件系统的操纵和控制能力；
- 7) **书写自由、使用灵活**
- 8) **可移植性好**：对硬件依赖低，(源代码) 可以移植到不同的计算机上执行

1.5 着手编程：从第一个程序做起

源程序：采用程序设计语言（如汇编语言、C语言等）的文字、符号（英文字母、数字、运算符等）编写的程序代码。

- ◆ **文本文件，可用文本编辑器编写。**
- ◆ **人可编辑、阅读。**
- ◆ **计算机不能直接执行，需要经过“翻译”（编译或解释）转换成可执行程序（二进制程序）才能执行。**

可执行程序：二进制机器指令程序，计算机可直接执行。



1、准备工具环境：

(1) 一台安装了Windows 操作系统的电脑（其它OS自备）

(2) 安装编程工具软件 **Code::Blocks**

Code::Blocks官网：<http://www.codeblocks.org/>

下载安装包，安装Code::Blocks，一定要**安装GCC编译器**。

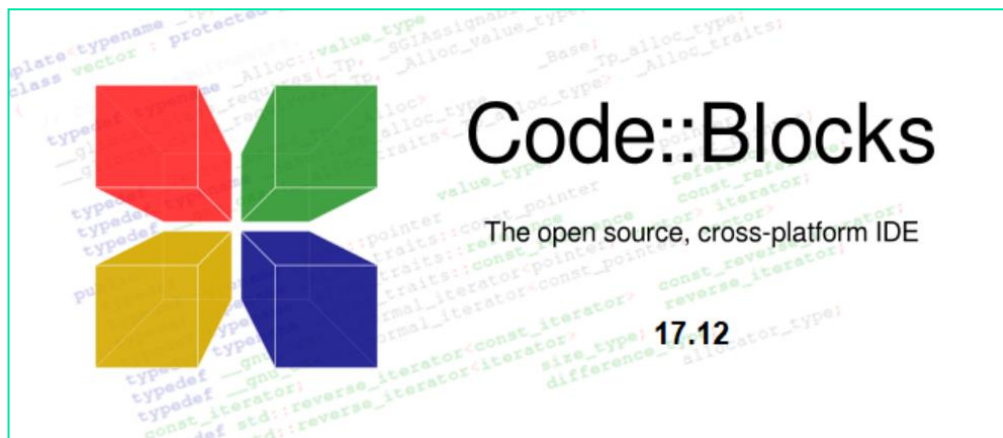


Code::Blocks中集成了**文本编辑器、编译器及调试器等工具**，集程序的**编辑、编译、链接、运行、调试**为一体，这样的开发工具称为**集成开发环境（IDE**，Integrated Development Environment）

其他IDE：**Dev C++、VS**等都可以，自行选用

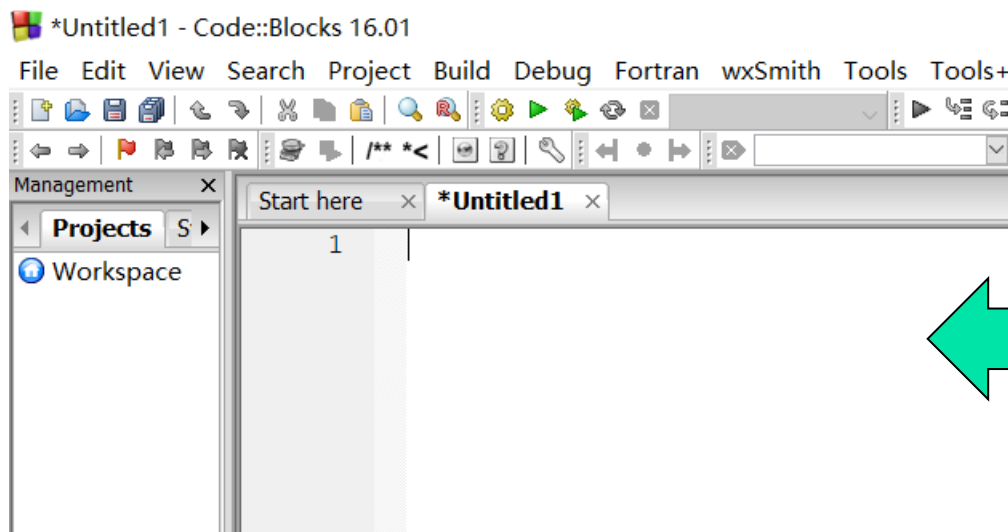
2、开始编程

(1) 执行Code::Blocks



(2) 创建一个空程序文件

主菜单→File →New →Empty File: 建立一个空文件



在**编辑区**输入
源程序代码

例1.1 输入你自己的名字的汉语拼音，要计算机问候一下自己并且输出这是你学习C语言的第一个程序的句子。

```
#include <stdio.h>
```

```
void show(char str[]);
```

```
int main(void)
```

```
{
```

```
    char name[20];
```

```
    printf("Input your name please!\n");
```

```
    gets(name);
```

```
    printf("Hello %s!\n", name);
```

```
    show(name);
```

```
    return 0;
```

```
}
```

```
void show(char str[ ])
```

```
{
```

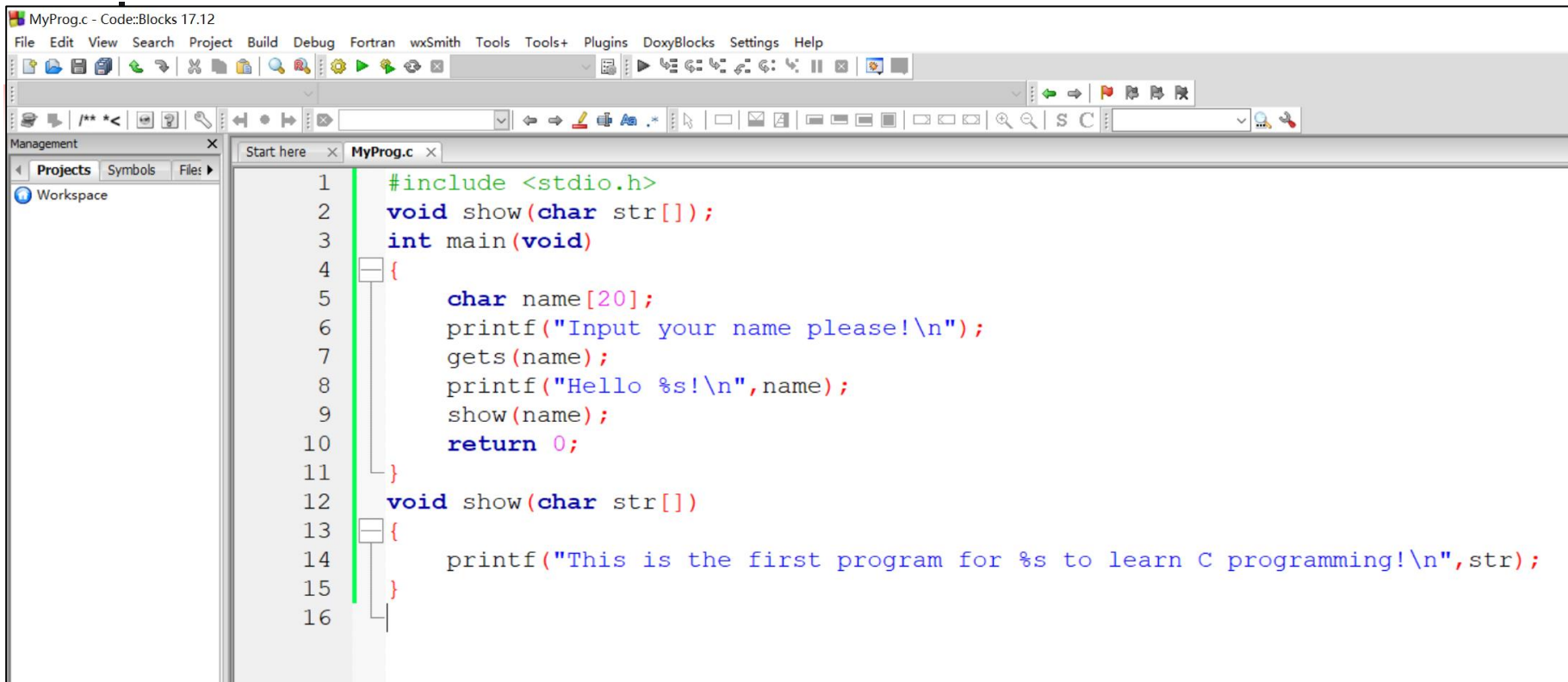
```
    printf("This is the first program for %s to learn C programming!\n",str);
```

```
}
```

➤ 每一行称为一条语句

➤ C程序由一条条“**语句**”组成

(3) 在编辑区编辑源程序：将程序文本输入到编辑区



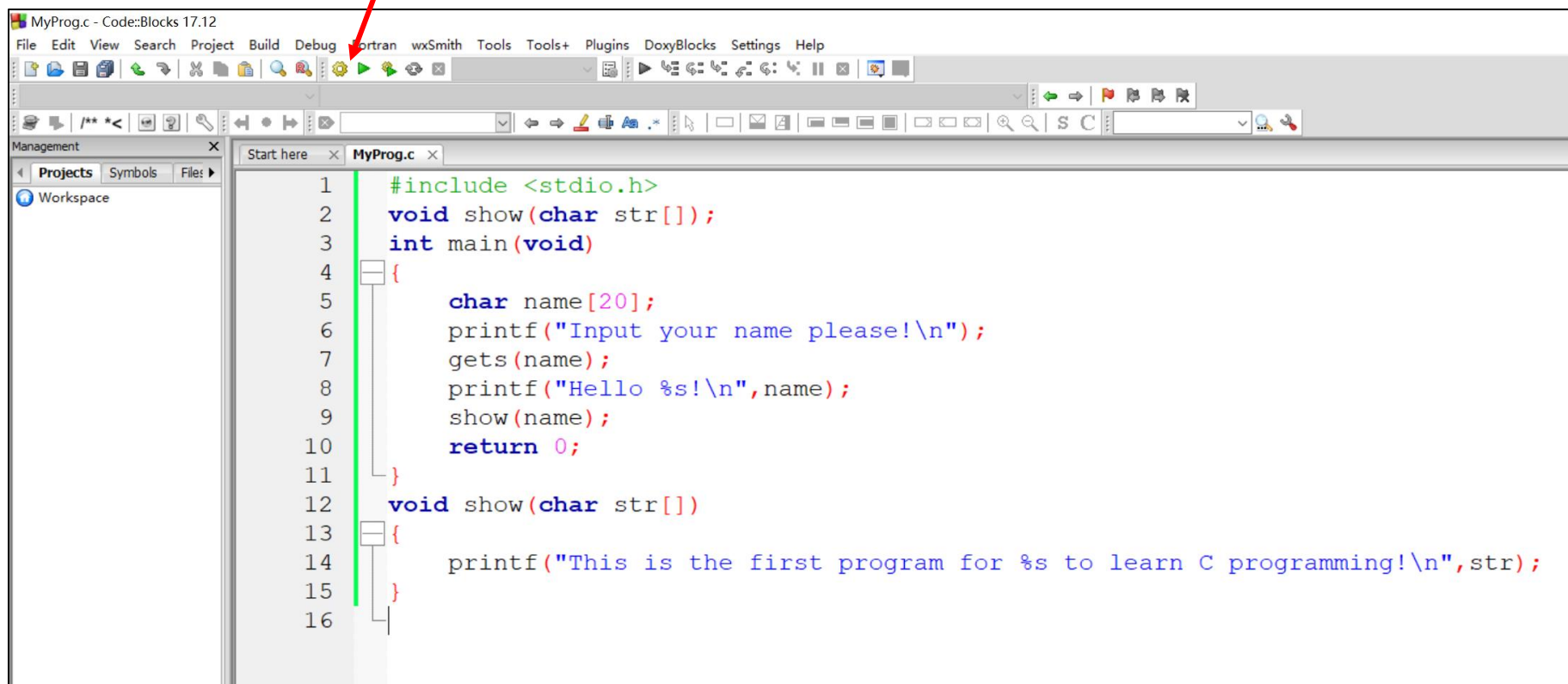
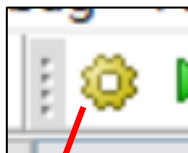
The screenshot shows the Code::Blocks IDE interface. The title bar reads 'MyProg.c - Code::Blocks 17.12'. The menu bar includes File, Edit, View, Search, Project, Build, Debug, Fortran, wxSmith, Tools, Tools+, Plugins, DoxyBlocks, Settings, and Help. The toolbar contains various icons for file operations, editing, and building. On the left, the 'Management' pane shows 'Projects' and 'Files' tabs, with 'Workspace' listed under Projects. The main editor window, titled 'MyProg.c', displays the following C code:

```
1  #include <stdio.h>
2  void show(char str[]);
3  int main(void)
4  {
5      char name[20];
6      printf("Input your name please!\n");
7      gets(name);
8      printf("Hello %s!\n", name);
9      show(name);
10     return 0;
11 }
12 void show(char str[])
13 {
14     printf("This is the first program for %s to learn C programming!\n", str);
15 }
16
```

要点：正确输入，注意英文字母的大小写，除字符串外，一般须使用英文符号，特别是不可使用中文标点符号，如“。”，“；”等。

(4) 编译源程序：将源程序翻译成机器语言程序

点击



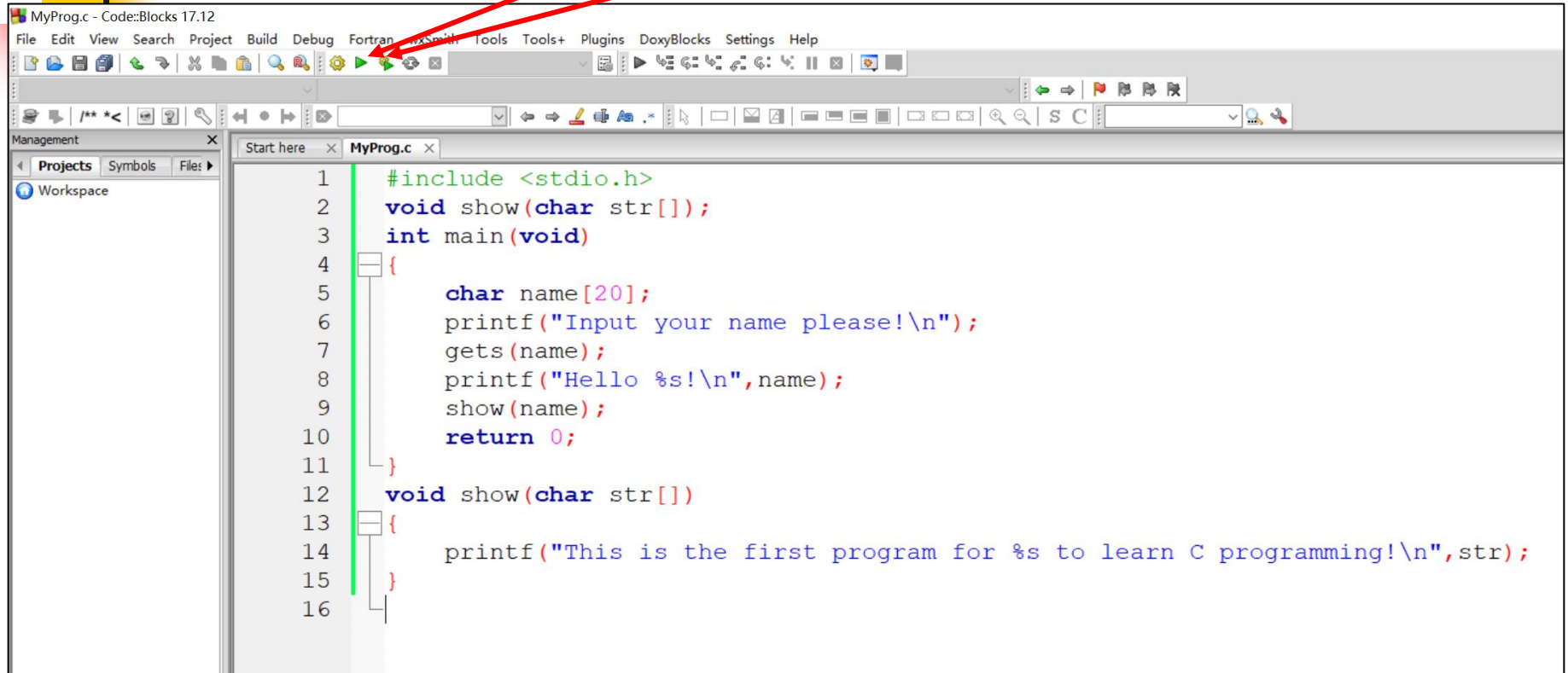
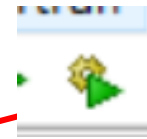
```
1  #include <stdio.h>
2  void show(char str[]);
3  int main(void)
4  {
5      char name[20];
6      printf("Input your name please!\n");
7      gets(name);
8      printf("Hello %s!\n", name);
9      show(name);
10     return 0;
11 }
12 void show(char str[])
13 {
14     printf("This is the first program for %s to learn C programming!\n", str);
15 }
16
```

注：如果编译报错，根据错误提示返回编辑区修改源程序，重复上述 (3)、(4)，直到成功通过编译，生成可执行程序。

(5) 运行: **执行程序**, 点击



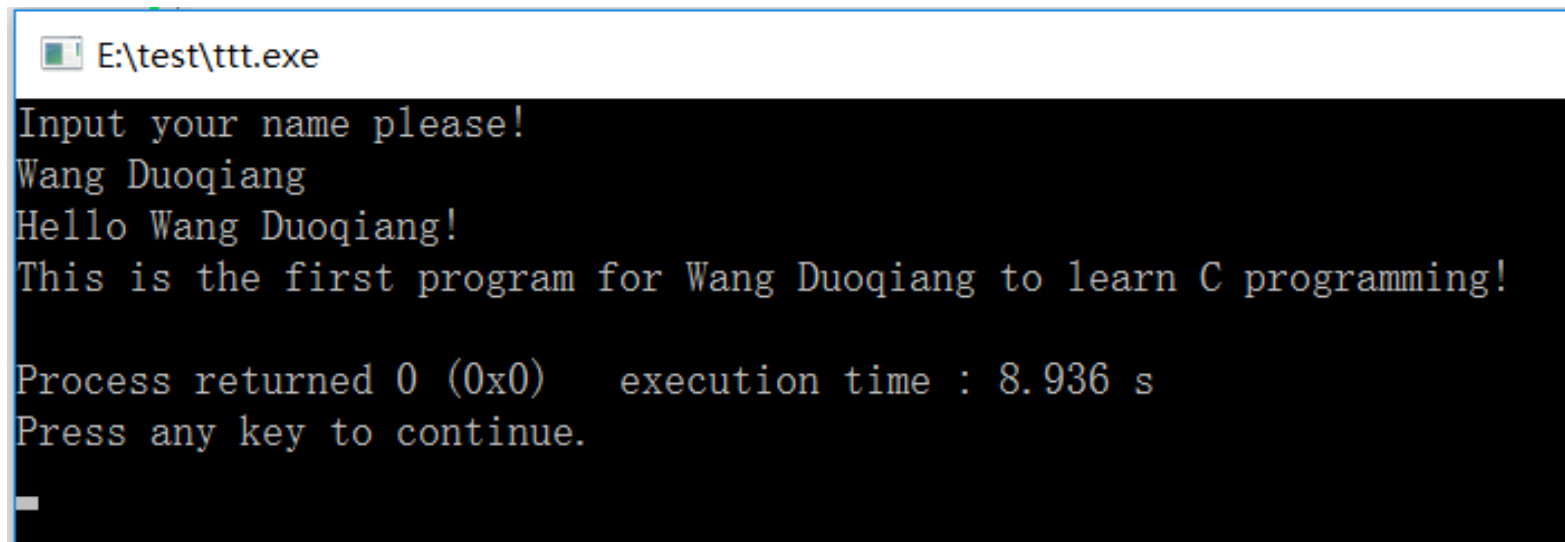
或

A screenshot of the Code::Blocks IDE interface. The title bar reads 'MyProg.c - Code::Blocks 17.12'. The menu bar includes File, Edit, View, Search, Project, Build, Debug, Fortran, wxSmith, Tools, Tools+, Plugins, DoxyBlocks, Settings, and Help. The toolbar contains various icons for file operations, editing, and execution. The 'Management' panel on the left shows 'Projects' and 'Workspace'. The main editor window displays a C program with line numbers 1 through 16. The code includes a header, a function declaration, a main function, and a helper function 'show'. A red arrow points from the text '执行程序' to the 'Run' icon (a green play button) in the toolbar.

```
1  #include <stdio.h>
2  void show(char str[]);
3  int main(void)
4  {
5      char name[20];
6      printf("Input your name please!\n");
7      gets(name);
8      printf("Hello %s!\n", name);
9      show(name);
10     return 0;
11 }
12 void show(char str[])
13 {
14     printf("This is the first program for %s to learn C programming!\n", str);
15 }
16
```

(6) 运行结果

- ◆ 运行时根据提示输入你的名字的汉语拼音。如果一切正确，则执行结果如图所示：



```
E:\test\ttt.exe
Input your name please!
Wang Duoqiang
Hello Wang Duoqiang!
This is the first program for Wang Duoqiang to learn C programming!

Process returned 0 (0x0)   execution time : 8.936 s
Press any key to continue.
_
```

- ◆ 而如果执行时有**报错**或显示的**结果与预期不符**，则表示程序有错误，返回编辑区仔细修改程序，重复(3)、(4)、(5)，直到程序能正确运行。

对程序的说明

```
#include <stdio.h>
```

```
void show(char str[]);
```

```
int main(void)
{
    char name[20];
    printf("Input your name please!\n");
    gets(name);
    printf("Hello %s!\n",name);
    show(name);
    return 0;
}
```

```
void show(char str[])
```

```
{
```

```
    printf("This is the first program for %s to learn C programming!\n", str);
```

```
}
```

【1】文件包含语句

#include: 文件包含命令

stdio.h: 外存中一个叫“stdio.h”的**头文件**的文件名

【2】show函数的**函数声明语句**（**函数原型**）

【3】main函数的**函数定义**

【4】show函数的**函数定义**

C语言基础：

1) 包含命令： **#include**

#include <stdio.h>的作用是什么？（第六章讲述）

现在只记着：编写C程序时，**第一条语句就写这句。**

2) 函数：引用数学函数的概念

如，设有数学函数 $f(x,y,a)=x+y+\sin(a)$

- ◆ **f**：函数名
- ◆ **(x,y,a)**：参数表，x、y、a是参数
- ◆ **x+y+sin(a)**：函数体——函数的计算公式
- ◆ **sin(a)**：在f函数中又调用了另外一个函数sin()
- ◆ **=**：给出函数f的定义

- ◆ 函数的使用称为**函数调用**:

设有变量p、z, 令

$$f(x,y,a)=x+y+\sin(a)$$

$$p=2, \quad z = \underbrace{f(p, 3, 90)}_{\text{函数调用}}$$

赋值

含义:

- ◆ 变量p赋予值2 $\longrightarrow$ **赋值**
- ◆ 然后用p、3、90调用函数f $\longrightarrow$ **函数调用**
- ◆ f函数使用当前参数的值、按照函数公式给出的计算方法计算出结果 $\longrightarrow$ **执行函数**
- ◆ 然后把结果赋给z $\longrightarrow$ **赋值**

C语言借用数学里“函数”概念，**将按照以下形式**
定义的程序段也称为函数：

函数名

参数表： str是参数名，char是参数的数据类型

void show (char str[])

函数体

```
{  
    printf("This is the first program for %s to learn C programming!\n",str);  
}
```

函数体： 由一对**花括号**界定，**由若干语句有序组成**，说明了实现函数功能要执行的运算。

main也是一个函数

函数头

如果函数没有参数，参数表里可为空或写void，但圆括号不能少

```
int main(void)
```

```
{
```

```
char name[20];  
printf("Input your name please!\n");  
gets(name);  
printf("Hello %s!\n",name);  
show(name);  
return 0;
```

```
}
```

main函数的
函数体

函数定义 = 函数头 + 函数体

通过return语句将结果“返回”

如果函数有**返回值**，在函数名前写出函数返回值的类型名（这里int表示main函数将返回一个整型数值）

show函数的调用

```
int main(void)
```

```
{
```

```
    char name[20];
```

```
    printf("Input your name please!\n");
```

```
    gets(name);
```

```
    printf("Hello %s!\n",name);
```

```
    show(name);
```

```
    return 0;
```

```
}
```

这三条语句
是什么?

用参数name调用show函数

实参：调用函数时使用的参数称为实参。如这里的
name，是调用show函数时使用的实参。

- ◆ 你自己定义的函数叫做“**用户自定义函数**”，如show函数；
- ◆ C语言还提供了**内部预定义好的函数**，这些函数不需要用户自己编写，需要时**只需调用即可**，这类函数称为“**系统函数**”或“**库函数**”。上述 **printf**、**gets**就是两个库函数

- ▣ **printf(...)**函数：将参数表里**双引号里面的内容**输出至屏幕

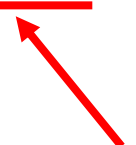
如：**printf("Input your name please!\n");**

- ▣ **gets(...)**函数：接收用户通过键盘输入一个字符串，并把输入的字符串保存到**参数数组**中。

如：**gets(name);**

这里**name**是调用gets函数使用的实参

```
int main(void)
{
    char name[20];
    printf("Input your name please!\n");
    gets(name);
    printf("Hello %s!\n",name);
    show(name);
    return 0;
}
```



这是什么?

也是函数调用

调用库函数

调用用户自定义函数show

调用库函数和调用用户自定义函数在形式上是一样的

如果函数没有返回值，函数名前写上**void**，表示该函数没有返回值。



```
void show(char str[ ])
```

```
{
```

```
    printf("This is the first program for %s to learn C programming!\n",str);
```

```
}
```



没有返回值的函数，函数体里可以没有return语句。

3) 变量：同样类似于数学里“变量”的概念，程序里需要定义**变量**来保存要处理的数据，且**变量的值可以被改变**。

本例中，要保存输入的姓名，而姓名是**字符串**数据，即由字符组成的字符序列，C语言用**字符数组**（一种变量形式）来保存字符串，所以定义了一个长度为20的字符数组name：

数组的长度

char name[20];

变量的数据类型 变量名称

▣ 使用时直接用**变量名**引用

如 用name作为参数调用gets函数：**gets(name);**

C语言变量的定义：

① 简单变量的定义

简单变量：只能保存一个**基本数据类型**的数据的变量。

简单变量的定义形式：**数据类型 变量名**；

◆ **数据类型**：说明变量是什么类型的

整型：**int**

浮点型（实数）：**float**、**double**

字符型：**char**

如：**int x, y;**

float score;

char flag;

double dist;

◆ **变量名**：变量名称，合法的标识符

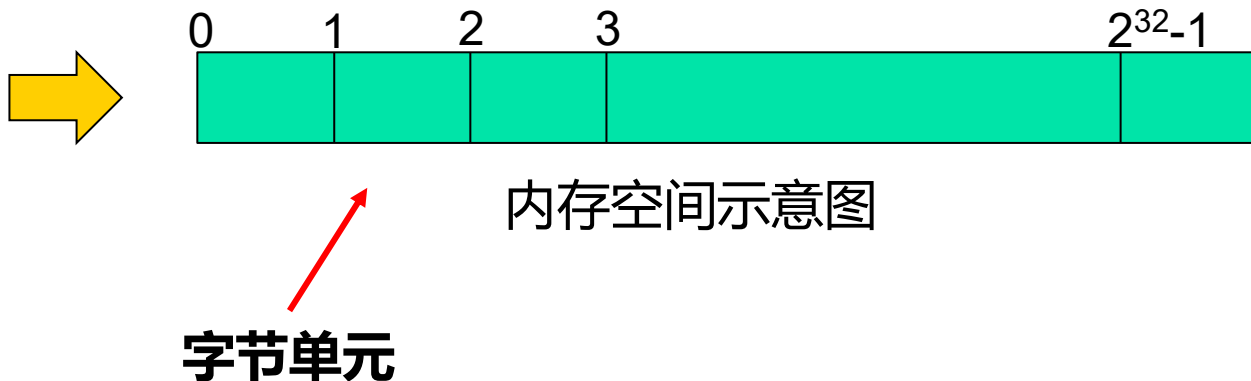
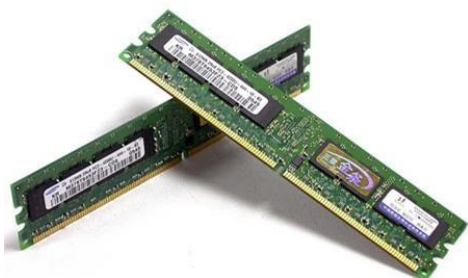
◆ **最后的分号不能少**

变量的定义，如： `int x=0x1234abcd;` **意味着什么？**

—— **和内存存储有关。**

◆ 认识内存：

内存，物理上是插在主板上的**内存条**，是用于存储的电子器件，但逻辑上，内存条被抽象**为由字节组成的存储空间**。



怎么理解内存空间



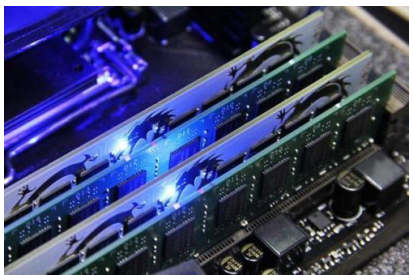
一个内存条相当
于一个书架



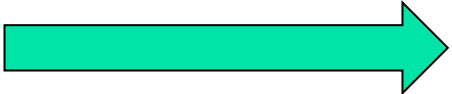
内存条由称之为“**字节**”
的存储单元组成

书架由**一格一格**的单元组成

内存中的一个**字节**相当于书架中的一**格**



多条内存条相
当于多个书架



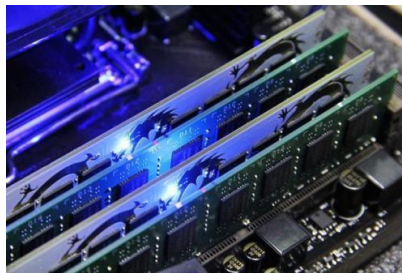
内存空间是所有**字节**单元的集合

内存单元的地址编号

书架格子编号：



- ◆ 从第一个书架开始编号，第一个书架的**第一个格子编号为0**，后续格子顺次编号为1, 2, ...。
- ◆ 第一个书架的格子全部编完后，第二、三...个书架的格子**接着编，编号都依次递增**。
- ◆ 若一个书架有N个格子，共有M个书架。则所有书架的格子的编号的范围是 **$0 \sim M * N - 1$ （线性编号）**。在这个范围内，**每个格子都有一个唯一的编号** ——可以把编号看作是**格子在书架空间中的地址**，编码范围看作是书架格子的**“地址空间”**

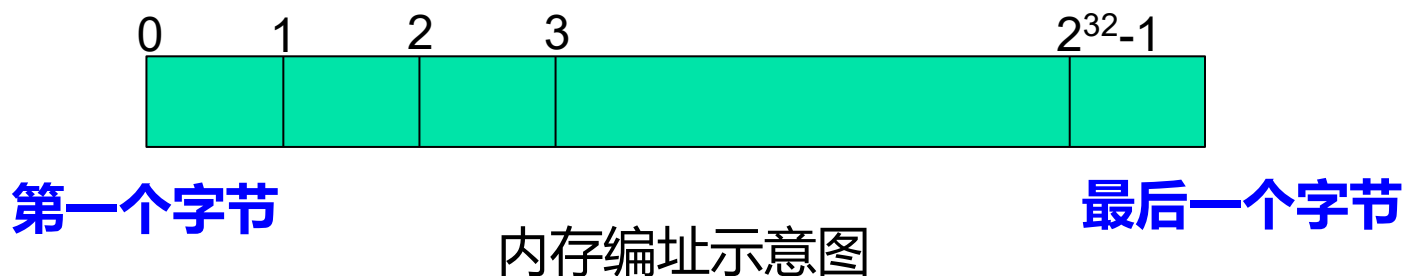


内存单元编号：

从第一个内存条的第一个字节开始编号（**起始编号为0**），对所有内存条的所有字节依次编号为0、1、2、...。这样，**每个字节都有一个唯一的编号**：

- ◆ 该编号是对应字节在内存中的地址，简称为**字节的内存地址**；
- ◆ 所有字节的地址集合称为**内存地址空间**。

假定内存空间中有4G个字节，则这些字节地址的编址范围为 $0 \sim 2^{32}-1$ ，是一个**线性的地址空间**，0对应最低地址， $2^{32}-1$ 对应最高地址。**每个字节有自己的一个唯一地址。**



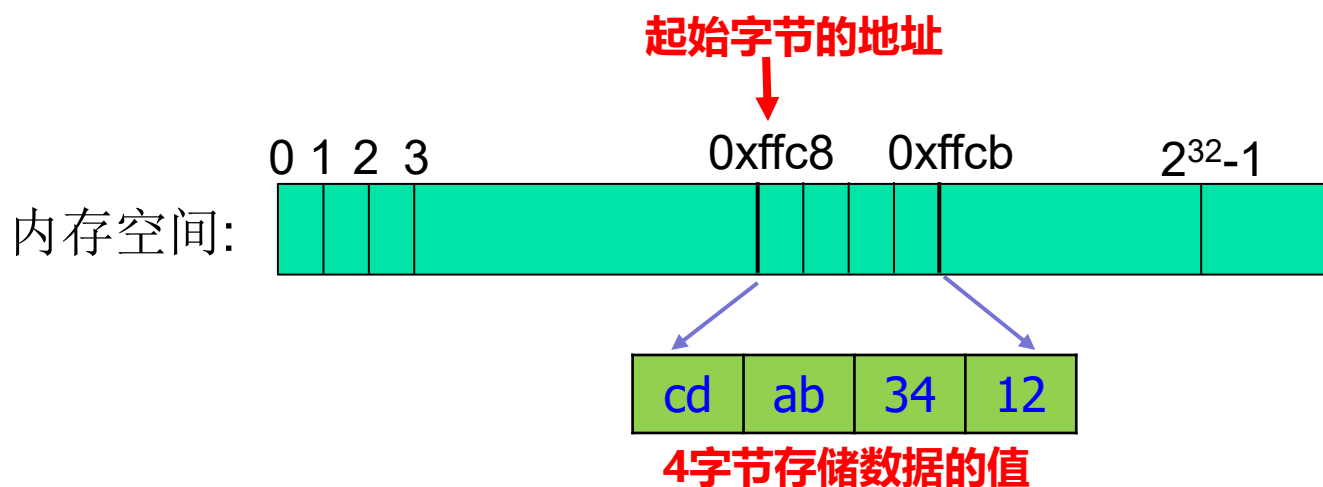
内存容量的单位：

- 一般用**B**表示字节 (Byte；相应地以**b**表示位，bit)
- 内存容量单位：**K**： 2^{10} ，1KB=1024B，
M： 1024K， 2^{20} ， 1MB=1024KB，
G： 1024M， 2^{30} ， 1GB=1024MB，
T： 1024G， 2^{40} ， 1TB=1024GB，
1024T？ P、E、Z等单位

一个数据在内存中通常用连续的1个或多个字节存储其值。

如果知道一个数据在内存中存储的起始字节，则从该字节开始，连续访问若干字节，就可访问到整个数据。

如，一个用4字节存储的数据：



- ◆ **字节中存放数据。字节是计算机中最小的存取单元，却不是最小的存储单位，最小的存储单位是“位” (bit), 一个字节包含8位。**

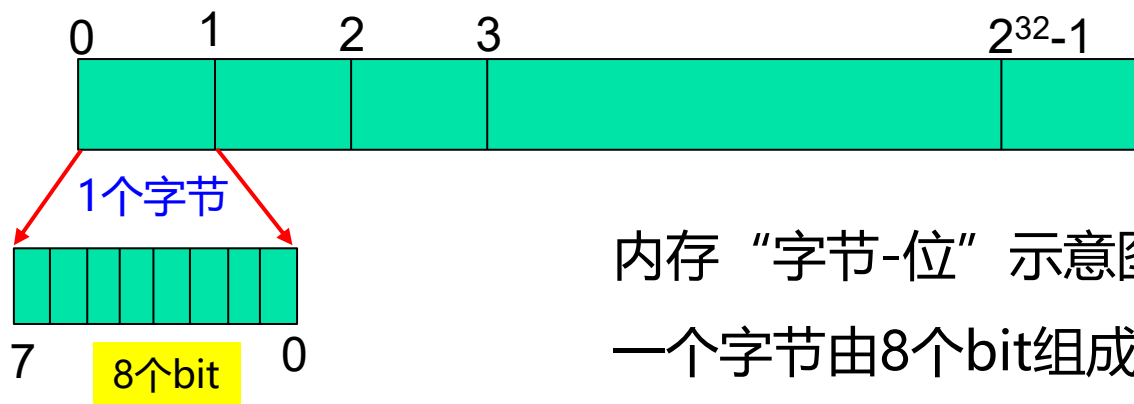
比方：书架的一个格子中又有8个小格子，每个小格子可存放一本书，则一个格子最多可以存放8本书。而图书管理员整理书架的时候是以**一格一组（最多8本书）**的方式存取书架中的书籍的。



计算机系统中，**每个字节有8个子单元（位）组成。每一位可以存放一个二进制的0或1（一位二进制数）。**

系统在访问内存单元时**以字节为单位、每8位为一组**进行读写的——**位虽是最小存储单元（数据单元），但读写数据必须以字节为基本单位进行，一次至少读写8位数据。**

内存示意图：



可见，虽然内存本质上是由连续的“位”组成的，但系统在组织管理内存时，是以**每8位为1个字节**的方式，将整个内存划分成**连续的字节空间**，并只针对字节进行编址和存取。

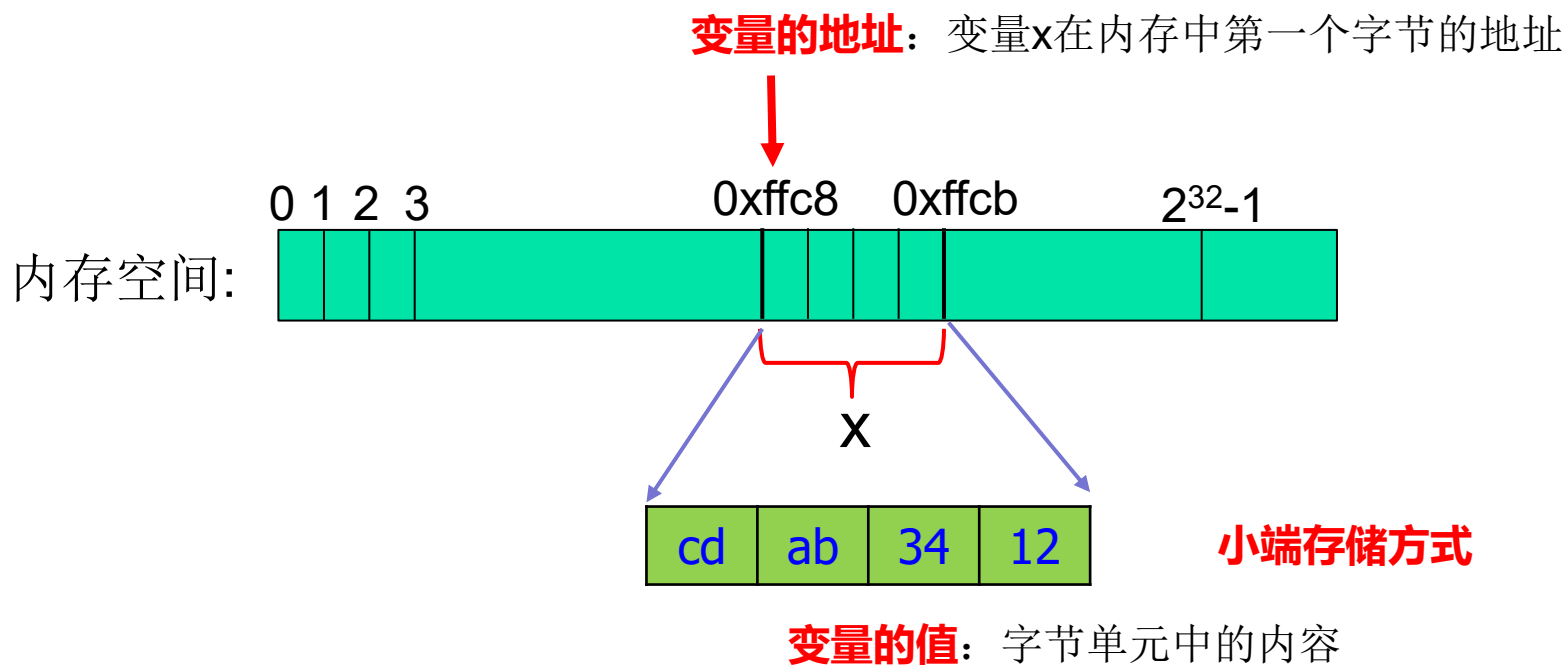
而“位”没有独立编址，不能单独进行直接访问。

要想对“位”进行存取，必须**借助特殊的位运算**。

声明变量意味着什么？

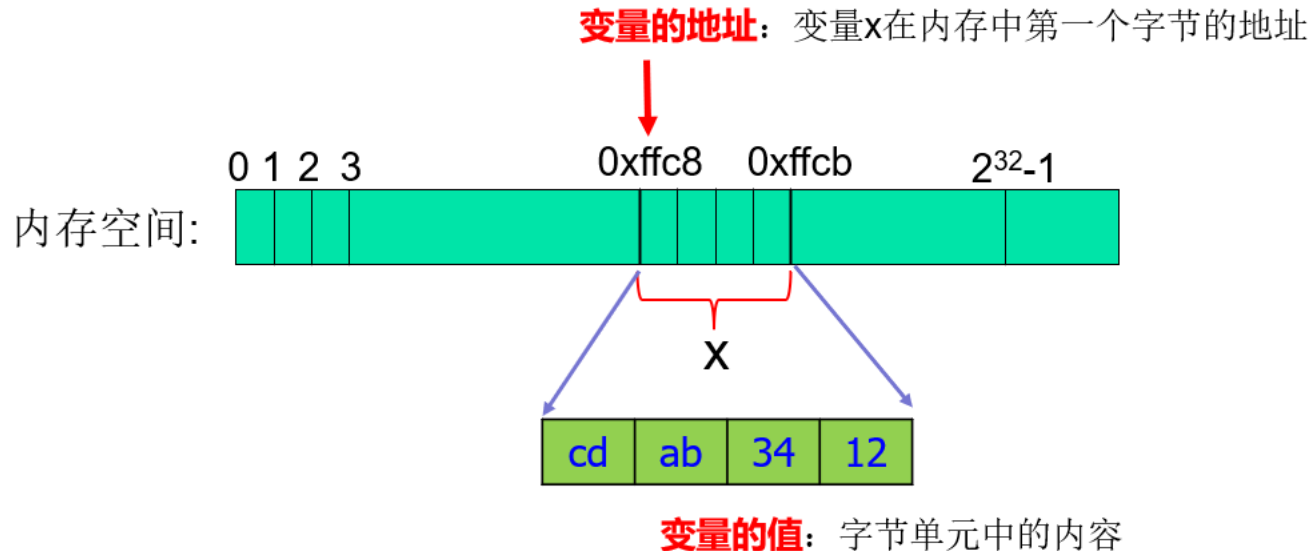
如： `int x=0x1234abcd;`

意味着： 系统为变量在内存中分配一个或多个连续的字节，用于保存变量的值。



注：在32位系统中，一个int型变量分配有4个内存字节保存其值

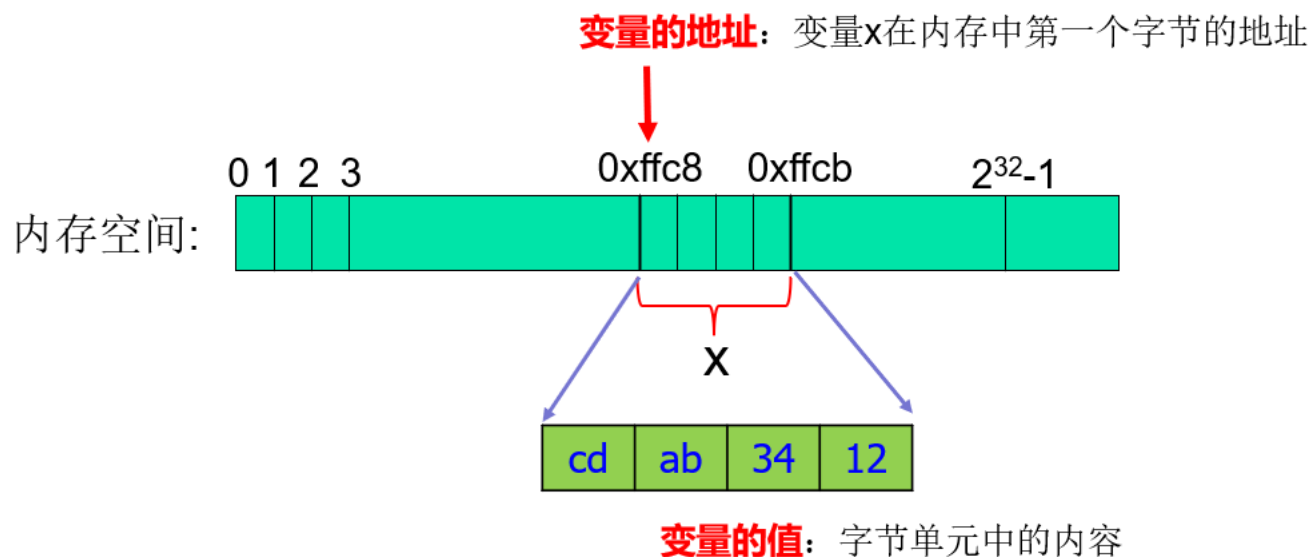
```
int x=0x1234abcd;
```



变量是数据在内存中所在单元的有名表示：

- ◆ **变量名：** 所在单元的名字，代表所占用的这1个或多个字节；
- ◆ **变量值：** 字节的内容，即字节中存储的0/1数字序列（二进制表示）；
- ◆ **变量的地址：** 变量所占字节中的第一个字节的内存地址

int x=0x1234abcd;



- ◆ 声明了变量后，就可以引用变量参与相关运算：

如： **(一般直接用变量名引用变量即可)**

y=x; **读： 获取x单元中的内容 (x的值)**

x=1000; **写： 向x单元中存入新内容 (赋新值)**

◆ 变量的类型：声明变量是必须说明其类型

(1) 不同类型的变量在内存中所分配的字节数是不同的。

- ◆ 标准整型 (**int**) : 4字节
- ◆ 长整型 (**long int**) : 4字节或8字节
- ◆ 字符型 (**char**) : 1字节
- ◆ 单精度浮点型 (**float**) : 4字节
- ◆ 双精度浮点型 (**double**) : 8字节

数据长度：一个某种数据类型的数据在内存中占用的**字节数**称为该类型的**数据长度**。

如，int的数据长度是4， char的数据长度是1。

同一类型的所有数据(变量、常量)都一样“长”。

(2) 不同类型的数据在内存中的**编码**表示不同。

- ◆ **整型** (**char**、**int**、**long int**) : 原码、补码、反码等;
- ◆ **浮点型** (**float**、**double**) : IEEE754编码。(第二章介绍)

(3) 不同类型的数据可参与的运算有所不同。

- ◆ 如 % 运算的操作数只能是整型数据, 浮点型数据不行。

为什么定义变量时一定要说明其类型? 主要是使编译器明确:

- ◆ 为这个变量分配几个字节来保存其值;
- ◆ 变量的数据以什么编码方式保存在字节中;
- ◆ 该变量/数据能参与什么运算。

进而翻译出正确的可执行程序。所以, **C语言要求定义变量时必须指定变量的数据类型。**

自学任务：进位计数制和数据的编码表示

一、进位计数制

- 1、十进制、二进制、八进制、十六进制的概念和数码表示
- 2、十进制数和二进制数的相互转换：整数转换、小数转换
- 3、二进制、八进制、十六进制数之间的相互转换

二、数据编码

- 1、**整数编码**：真值、原码、反码、补码

计算机内的整数编码表示（补码）

- 2、**浮点数编码**：IEEE754标准（单精度、双精度）

- 3、**字符编码**：

- ◆ **ASCII码**

- ◆ 汉字编码（输入码、区位码、国标码、机内码）

② 数组

数组：是有限个**同类型数据**的集合，数组中的每个数据称为一个**元素**，一个数组中的所有元素必须是相同类型的。

数组的定义形式：**数据类型** **数组名** [**长度**];

↑
数组元素的数据类型

- ◆ 方括号必须有。
- ◆ 长度须为正整数，表示数组中**元素**的个数，称为**数组长度**。

如：**int x[10];**

定义了一个有10个元素的整型数组，数组长度为10，数组（元素）类型是int型，即数组x中可以存放10个int型数据。

再如：**char name[20];**

float f[3];

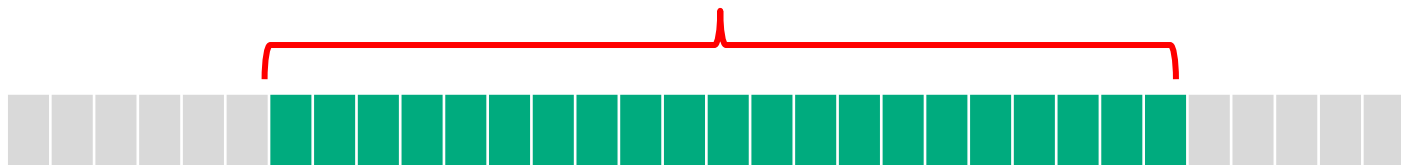
double t[8]; 等

数组的存储空间分配：

规则： 在内存中连续分配 **元素个数 × 数据类型长度** 个字节，并分成 **元素个数** 个**段**，每段存放一个该类型的数据（**元素**）。

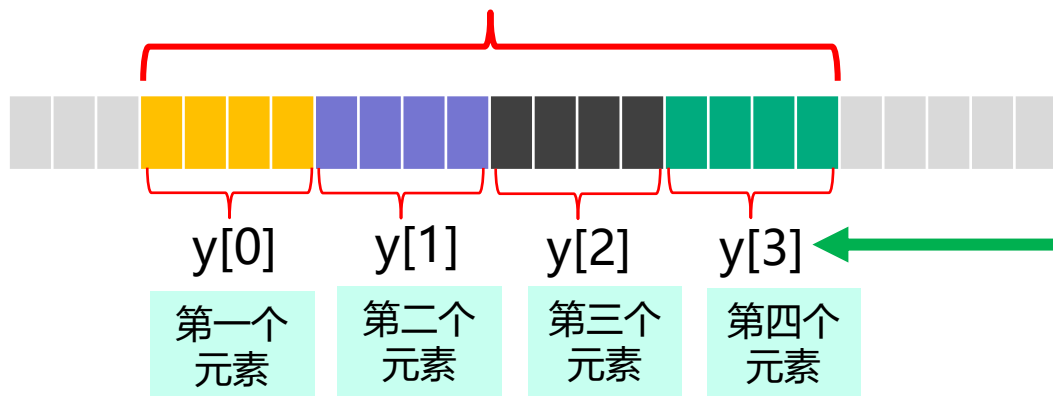
如：char name[20];

连续分配20个字节，每个元素1个字节



如：int y[4];

连续分配16个字节，每个元素4个字节



数组名[下标]： 引用数组元素

注：**下标从0开始**

◆ 再定义一个函数

求两个非负整数 x 和 y 的最大公约数 (**辗转相除法**):

如果 y 等于0, 则 x 和 y 的最大公约数等于 x ; 否则 x 和 y 的最大公约数等于 y 与 x 除以 y 的余数的最大公约数。

设 **gcd(x, y)** 是求 x 和 y 最大公约数的函数。

如: $\text{gcd}(4, 0)=4$;

$$\text{gcd}(12, 8)=\text{gcd}(8, 4)=\text{gcd}(4, 0)=4$$

定义C函数 **gcd(x, y)**, 求其参数 x 和 y 的最大公约数:

定义gcd(x,y)函数:

参数声明和普通变量一样，要说明其类型和名称

```
int gcd(int x, int y)
{
    int r;
    while(y!=0)
    {
        r=x%y;
        x=y;
        y=r;
    }
    return x;
}
```

- ◆ **r**: 定义在gcd函数里的一个**局部变量**。
- ◆ **while**: **循环语句**，表示反复执行其后花括号里的语句，直到**循环条件**不成立。
- ◆ **%**: 求模（余数）运算。
- ◆ gcd函数有返回值。

循环体: y不等于0时，重复：计算x除以y的余数、置x等于y、y等于x除以y的余数，直到y等于0结束循环。

注: 这是“否则x和y的最大公约数等于y与x除以y的余数的最大公约数”的**程序实现**。

理解 **gcd(x,y)** 函数的计算过程:

```
int gcd(int x, int y)
{
    int r;
    while(y!=0)
    {
        r=x%y;
        x=y;
        y=r;
    }
    return x;
}
```

不行! **x**的值在计算 $x\%y$ 之前不能发生变化, 否则就不能正确计算余数。

r: 临时变量, 用于暂存一个值, 保证后续计算的正确 (这里是先保存当前 $x\%y$ 的值, 然后把y的赋给x以更新x, 再把r赋给y以更新y, 这才是正确的计算过程!)。

思考: (1) 这里能不能写成如下形式?

```
x=y;
y=x%y;
```

任何运算都是有先后次序的, 不能随意颠倒或想当然。

理解gcd(x,y)函数的计算过程（续）：

```
int gcd(int x, int y)
```

```
{
```

```
    int r;
```

```
    while(y!=0)
```

```
    {
```

```
        r=x%y;
```

```
        x=y;
```

```
        y=r;
```

```
    }
```

```
    return x;
```

```
}
```

对象（变量）是同一个，但值不同。经过上面的计算，一般情况下x的值已发生变化，返回时x的值与入口时x的值不一定相等。

思考：(2) 这个x是原先（入口时）的x吗？

调用gcd函数：

```
int gcd(int x, int y)
```

```
{
```

```
    int r;
```

```
    while(y!=0) { r=x%y; x=y; y=r; }
```

```
    return x;
```

```
}
```

```
int main( )
```

```
{
```

```
    int g;
```

```
    g=gcd(12, 8);
```

```
    printf("12和8的最大公约数是%d", g);
```

```
}
```

可以将多条语句写在一行里



调用gcd函数：求12和8的最大公约数（这里12和8就是实参），并将函数的返回值赋给变量g。



可以用中文

4) 关于printf函数的进一步说明

printf: 格式化输出函数 —— 按指定的样式输出信息

```
printf("Input your name please!\n");
```

```
gets(name);
```

格式串

```
printf("Hello %s!\n", name);
```

格式转换符

- ◆ **格式串中没有格式转换符时**，printf函数仅简单地将参数里双引号之间的内容输出（不含双引号）。
- ◆ **格式串中包含格式转换符时**：先用**后面参数的值**替换格式串中**由%引导的格式转换符**（如：用name字符串的内容替换"%s"的出现），然后再输出整个内容（不含双引号）。

再如：

```
void show(char str[ ])
```

参数字符数组，代表要传进来一个字符串

```
{  
    printf("This is the first program for %s to learn C programming!\n", str);
```

用str的内容替换%s

```
}  
  
This is the first program for Wang Duoqiang to learn C programming!
```

```
g=gcd(12, 8);
```

```
printf("12和8的最大公约数是%d", g);
```

%d用于输出整数数据，这里用g的值替换%d

```
12和8的最大公约数是4
```

5) scanf函数

scanf: 格式化输入函数——按指定的样式接收从键盘输入的数据

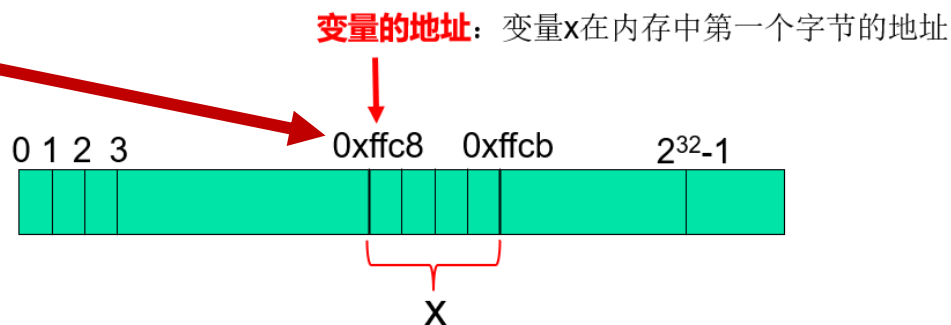
`int x;`

`scanf("%d", &x);`

格式串

格式符

接收变量
(的地址)



- ◆ **"%d"** : 格式串, 其中的**格式符%d**表示要等待输入一个整数。
- ◆ **&x**: 接收输入的变量, scanf要求用**接收变量的地址**作为参数。
- ◆ **&**: 取地址运算符, &x表示获取**变量x的地址**。

如上图所示, **&x的结果等于0xffc8**

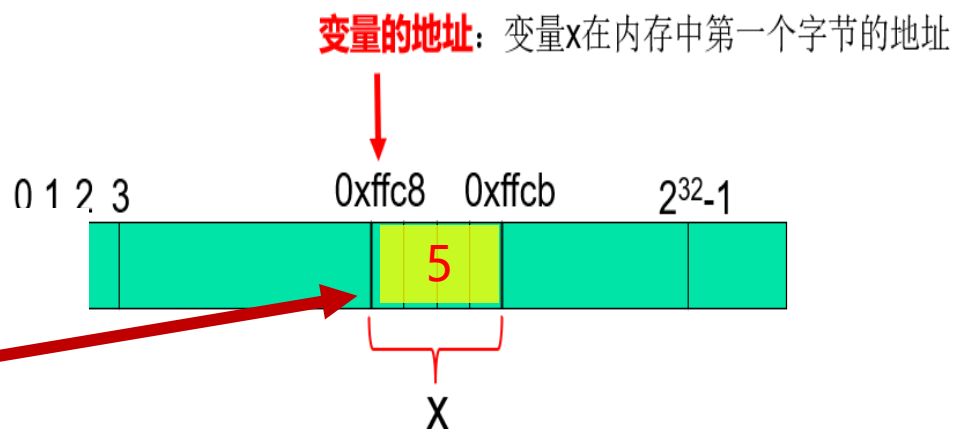
scanf函数的使用

```
int x;
```

```
scanf("%d", &x);
```

设有输入：

5



改进gcd:

从键盘输入两个整数，将其值保存到整型变量x和y中，然后求x和y的最大公约数。

如何从键盘输入两个整数？

```
int x, y;
```

```
scanf("%d%d", &x, &y);
```

然后用x和y调用以前的gcd函数即可。

改进的程序:

```
#include <stdio.h>

int gcd(int x, int y)
{
    int r;
    while(y!=0) { r=x%y; x=y; y=r; }
    return x;
}
```

请输入两个整数: 12 8
12和8的最大公约数是4

```
int main( )
```

```
{
```

```
    int x, y, g;
```

```
    printf("请输入两个整数: ");
```

```
    scanf("%d%d", &x, &y);
```

```
    g=gcd(x, y);
```

```
    printf("%d和%d的最大公约数是%d", x, y, g);
```

```
}
```

用变量x和y的地址作为
scanf函数的实参

用变量值的地方直接用变
量名, 不要再用变量地址

常用的格式转换符

◆ **%d**: 输入、输出整数

```
int x;  
scanf("%d", &x);
```

◆ **%f**: 输入、输出浮点数

```
float f=1.5;  
printf("%f", f);
```

◆ **%c**: 输入、输出单个字符

```
char ch='A';  
printf("%c", ch);
```

◆ **%s**: 输入、输出字符串

```
char name[20];  
scanf("%s", name);
```

6) C程序的执行

一个C语言程序由若干**函数**构成，包括**main函数**、**用户自定义函数**和**库函数**。

- ◆ **main**函数是一个特殊的函数，每个C程序都必须有且仅有一个main函数，它是整个程序的“入口点”。
- ◆ **程序的执行从main开始**：从main函数里面的第一条可执行语句开始，按规定的顺序依次执行函数体里的语句；
- ◆ 当遇到对其它函数的调用时，则转向这些函数里执行（包括用户自定义函数和库函数）。在执行完函数里面的指令后**退出函数**，并**返回到之前调用它的地方**继续执行后续语句。
- ◆ **main函数既是起点也是终点**，当程序执行完main函数的最后一条语句后，结束于右花括号处并返回操作系统。

如: #include <stdio.h>

void show(char str[]);

int main(void)

{

变量声明语句，非可执行语句，不需要“执行”

char name[20];

printf("Input your name please!\n");

① 第一条可执行语句，从这里开始“执行”：调用printf函数输出一行信息

gets(name);

② 第二条可执行语句：调用gets函数等待输入姓名

printf("Hello %s!\n",name);

③ 第三条可执行语句：输出一行信息

show(name);

④ 第四条可执行语句：调用show函数，转show内部执行

return 0;

⑦ main函数的最后一条可执行语句，执行完后main函数就执行完了

}

⑧ main函数结束，整个程序执行结束

void show(char str[])

{

⑤ 进入show函数，执行printf函数，输出一行信息

printf("This is the first program for %s to learn C programming!\n",str);

⑥ show函数执行结束，然后返回main函数

再如：

```
#include <stdio.h>
```

```
int gcd(int x, int y)
```

```
{
```

变量声明语句，非可执行语句，不需要“执行”

```
int r;
```

```
while(y!=0) { r=x%y; x=y; y=r; }
```

② gcd函数的第一条可执行语句：执行计算

```
return x;
```

③ gcd函数的第二条可执行语句：返回计算结果

```
}
```

④ gcd函数执行结束，返回main函数

```
int main( )
```

```
{
```

变量声明语句，非可执行语句，不需要“执行”

```
int g;
```

```
g=gcd(12, 8);
```

⑤ 从gcd函数返回后，继续执行main函数中的后续操作：将gcd函数的返回值赋给变量g

```
printf("12和8的最大公约数是%d", g);
```

⑥ main函数的第二条可执行语句：输出信息

```
}
```

⑦ main函数执行结束，整个程序执行也就结束了

转gcd函数执行

① 第一条可执行语句，从这里开始“执行”：调用gcd函数

7) 从源程序到可执行程序



源程序：程序员编写，由普通文字符号组成，**文本文件**。

预编译后的程序：计算机（预编译器）自动处理，得到系统**内部使用的文本文件**，对程序员一般不可见。

编译：将（预编译后）的程序翻译成**可链接目标代码文件**，**二进制文件**，分模块生成，不完整，**还不能执行**。

链接：将可链接目标代码文件链接起来，生成**可执行目标代码文件**（可执行文件），**二进制文件**，完整，**可执行**。

执行：运行程序，得到结果。

1.6 如何使用计算机进行问题求解

一般遵循以下步骤：

- 1) **问题定义**：明确问题是什么
- 2) **设计算法**：明确求解问题的**方法和步骤**
方法：使用什么样的运算和操作
步骤：运算和操作的顺序（**按序进行，一般不能颠倒**）
- 3) **编制程序**：编写源程序、调试，使程序能够正确执行
- 4) **运行程序**：运行调试正确的程序，获取结果

注：在上述过程中，如果 3) 时发现算法错误，或 4) 时发现结果有问题，则一般需要返回 2)、3) 进行修正，直到能得到可以正确求解问题的程序。

1.7 算法及其表示

语言只是工具，而算法是问题求解的**核心**和**灵魂**。

1、什么是算法？

算法是解题方法准确而完整的描述，由**一系列指令**组成。这些指令不仅规定了求解问题所需要的**运算**，也规定了求解问题的**步骤**，按照这些步骤**依次执行**规定的运算就可以得到问题的答案。

2、算法的五个重要性质：

确定性：算法使用的每种运算必须要有确切的定义，不能有二义性。

能行性：算法中有待实现的运算都是基本的运算，原理上每种运算都能由人用纸和笔在“有限”的时间内完成。

输入：每个算法都有0个或多个输入。

输出：一个算法产生一个或多个输出，这些输出是同输入有某种特定关系的量。

有穷性：一个算法总是在执行了有穷步的运算之后终止。

3、算法的表示

(1) 算法≠程序

- ◆ 算法是在编写具体程序前给出的关于解题方法和步骤的**抽象描述**。而程序是用某种程序设计语言**具体实现**了的算法。
- ◆ 算法不需要用实际的程序设计语言描述。

4、算法表示的方法

- ◆ 自然语言
- ◆ 实际程序
- ◆ 伪代码
- ◆ 流程图

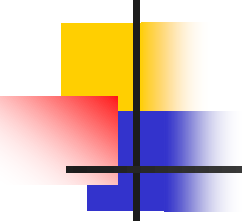
(1) 用自然语言描述算法

如求两个非负整数 x 和 y 的最大公约数的**辗转相除法**：

描述1：如果 y 等于0，则 x 和 y 的最大公约数等于 x ；否则 x 和 y 的最大公约数等于 y 与 x 除以 y 的余数的最大公约数。

描述2：（更规范一些）

- ① 输入两个整数，记为 a 和 b 。
- ② 如果 b 等于0，则 a 和 b 的最大公约数就是 a ，输出 a ，算法结束；否则
- ③ 计算 a 除以 b 的余数 r ，并令 a 等于 b ， b 等于 r ，然后重复步骤②和③。



优点：易读、易理解

缺点：规范性差，自然语言二义性强，容易有歧义。

(2) 用实际程序描述算法

用一种**具体的程序设计语言**，按照语言的严格规定描述算法。

如**辗转相除法**：

```
int gcd(int a, int b)
{
    int r;
    while(b!=0) {
        r=a%b;
        a=b;
        b=r;
    }
    return a;
}
```

优点：直接用程序描述，马上可用。

缺点:

- ◆ **太具体、太琐碎。** 需要注明每个变量的类型，语法细节多，容易出语法错。
- ◆ **通用性差。** 变量指定类型，同样的操作，针对不同类型的数据需要设计不同的算法。如加法，有**整数的加法**、**浮点数的加法**，规则一样，但如果写成程序，就要分开写：

```
int iadd(int x, int y)
{
    return x+y;
}
```

```
float fadd(float x, float y)
{
    return x+y;
}
```

(3) 用伪代码描述算法

算法描述尽量“**抽象**”一些，“**泛化**”一些，使其尽量有一定的通用性，不管以后使用哪种程序设计语言具体实现、处理什么样的数据，尽可能“适用”。

伪代码：程序语言+自然语言+数学符号混用，易读、易理解、规范。

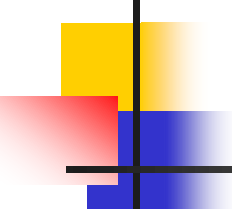
如：

用 $\leftarrow$ 代替 $=$ ，更清楚地表示出赋值的方向
(将右侧表达式的值赋给左边的变量)

```
function add(a, b)
{
     $z \leftarrow a + b;$ 
    return z;
}
```

关键字**function**说明add是个函数

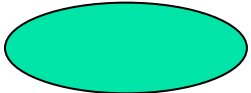
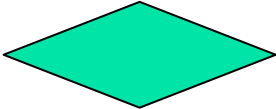
不指定具体的数据类型，使其不管是对整数，还是浮点数都适用。



需要注意：伪代码不是实际的程序，不能直接运行。必须用实际的程序设计语言（如C语言）编写出对应的程序，才能放到计算机中运行。

(4) 用流程图描述算法

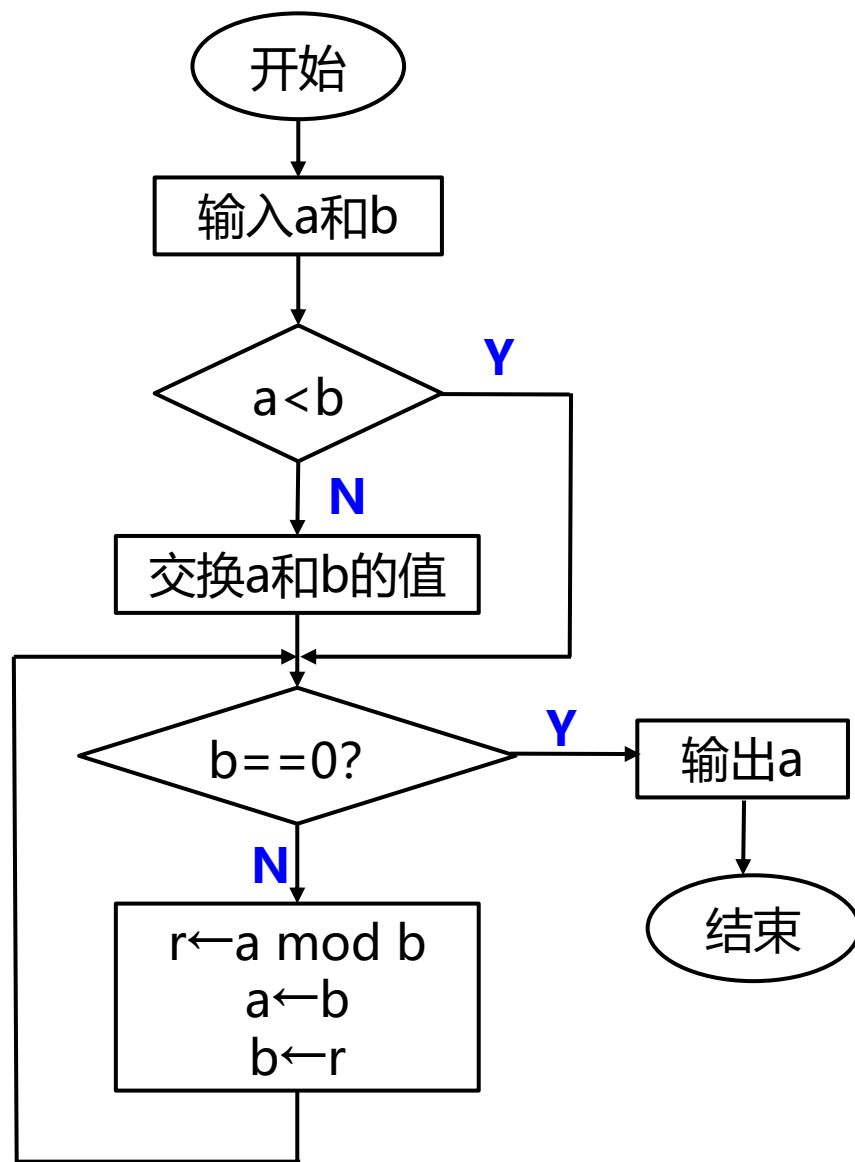
流程图用特定的**图型符号**来描述算法的每一步骤以及这些步骤之间的相互联系。**流程图的图形符号**:

- ◆ **起止框**: 椭圆框  表示处理流程的开始或结束
- ◆ **处理框**: 矩形框  框内用文字描述该步要做的操作
- ◆ **判断框**: 菱形框  框内标注判别条件, 根据条件**真/假(是/否)**选择执行后续的操作
- ◆ **流程线**: 带箭头的线段 (折线段)  连接不同的框, 箭头表示处理的先后顺序。
- ◆ **连接点**: 正圆框  框内写标号, 需要分块画图时, 连接点表示流程相连的位置

例 用流程图描述下面的算法

(辗转相除法)

- ① 输入两个整数，记为a和b。
- ② 如果 $a < b$ ，则交换二者的值；
否则不变。
- ③ 如果b等于0，则a和b的最大公约数就是a，输出a，算法结束；否则，
- ④ 计算a除以b的余数r，并令a等于b，b等于r，然后重复步骤③和④。



1.7 学习C语言与程序设计的方法

现在还是**菜鸟**的你，如何学会、掌握、以及熟练使用C语言？

从四个方面进行学习和训练：

首先，要学习并理解C语言的**语法和语义**；

其次，要学习并掌握一些**基本数据结构和常用算法**；

第三，要学习并熟悉C语言的**集成开发环境**；

第四，能熟练使用C语言提供的**各种库函数**。

最后的最后：怎么成为编程高手？

何以解忧，唯有多编！

代码量：学完本课程 ≥ 2000 行，毕业的时候 ≥ 20000 行

刷题：头歌的练习题、洛谷等平台上的题目



本章小结

- ◆ 电子计算机发展的历程
- ◆ 冯·诺依曼计算机系统的基本组成和工作原理
- ◆ 程序设计语言和程序设计
- ◆ C语言的产生与发展、标准化、特点
- ◆ 编程初步
- ◆ 用计算机进行问题求解的一般方法
- ◆ 算法的概念、重要性质、表示方法

课后作业

- ◆ 阅读教材第一章，结合课件理解相关内容
- ◆ 预习第二章